

Release 1 Data Model, API Contracts, and Background Job Specification

1. Executive data and API summary

Release 1 should use a relational PostgreSQL model in which customers, documents, attempts, reviews, warnings, approved policy versions, audit events, usage, email delivery, and durable jobs have explicit ownership and enforceable relationships. Candidate extraction data remains immutable and separate from editable review decisions. Approval copies the final reviewed values into dedicated immutable policy-version tables. FastAPI exposes versioned REST resources and remains the authority for authorization, validation, concurrency, idempotency, and state transitions. Long-running work is represented by durable PostgreSQL jobs claimed through leases by a separate worker. Private files remain in Supabase Storage and are accessed only through short-lived backend-authorized upload or download permissions.

One-sentence definition: A provider-neutral relational and REST contract that preserves every document-processing attempt, supports controlled human review, and creates immutable approved policy versions through transactionally safe backend actions.

Recommended persistence strategy

- PostgreSQL as the authoritative business store
- Private Supabase Storage for PDFs and large artifacts
- Relational tables for core business truth
- JSONB only for versioned configuration snapshots, evidence geometry, diagnostic metadata, immutable demo templates, and provider-neutral structured values where a scalar type varies by field
- Immutable candidates and policy versions
- Mutable review state with append-preserved material edits
- A separate current-policy pointer rather than mutating approved versions
- PostgreSQL-backed jobs with leases and execution history
- Generic idempotency records for high-consequence HTTP actions

Recommended API style

- REST-oriented `/api/v1` resources
- JSON requests and responses
- Bearer identity issued by Supabase Auth
- FastAPI object-level authorization
- Cursor pagination for timelines and operational lists
- Integer resource versions exposed as opaque concurrency tokens
- `Idempotency-Key` header for repeat-sensitive actions
- Stable machine-readable error codes
- Separate commands for approval, rejection, Technical Retry, Reprocessing, and Stored Result replay

Recommended background-job style

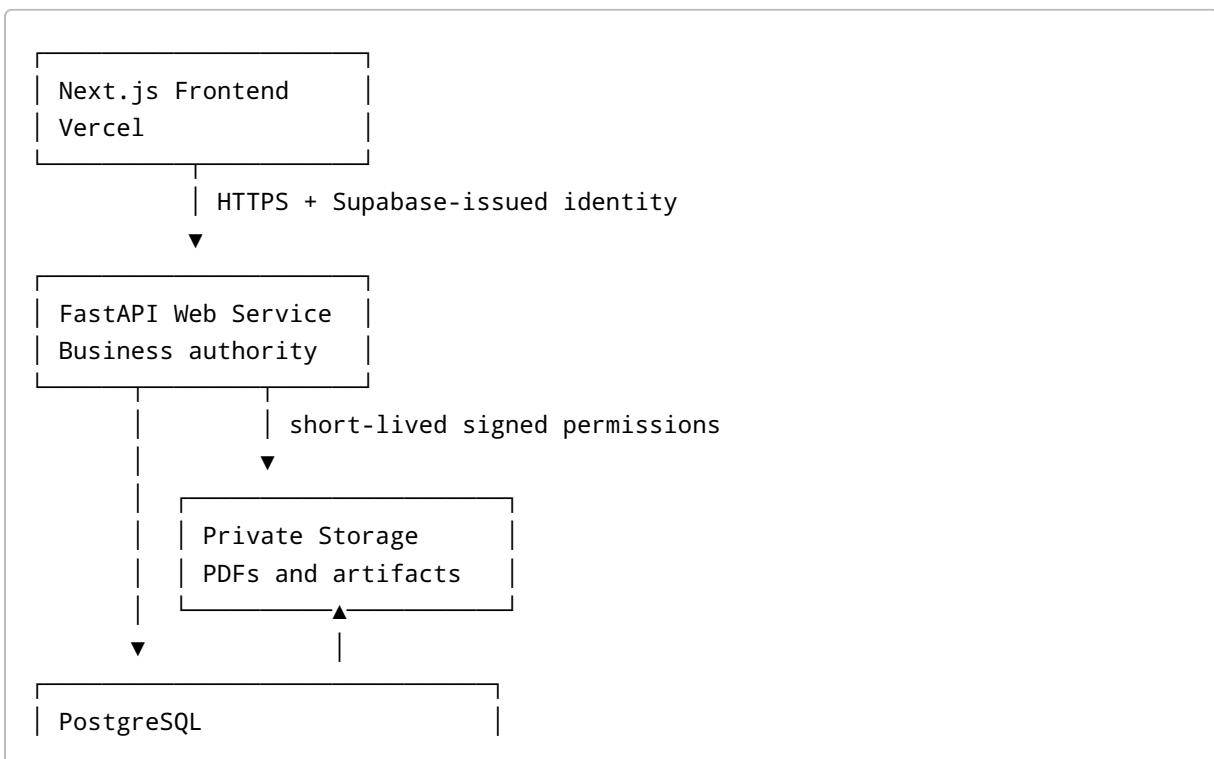
Use one orchestration job per processing attempt. The job advances durable processing stages and commits each completed stage separately. Email, cleanup, demo expiry, stale-attempt detection, and usage-reservation release use separate job types.

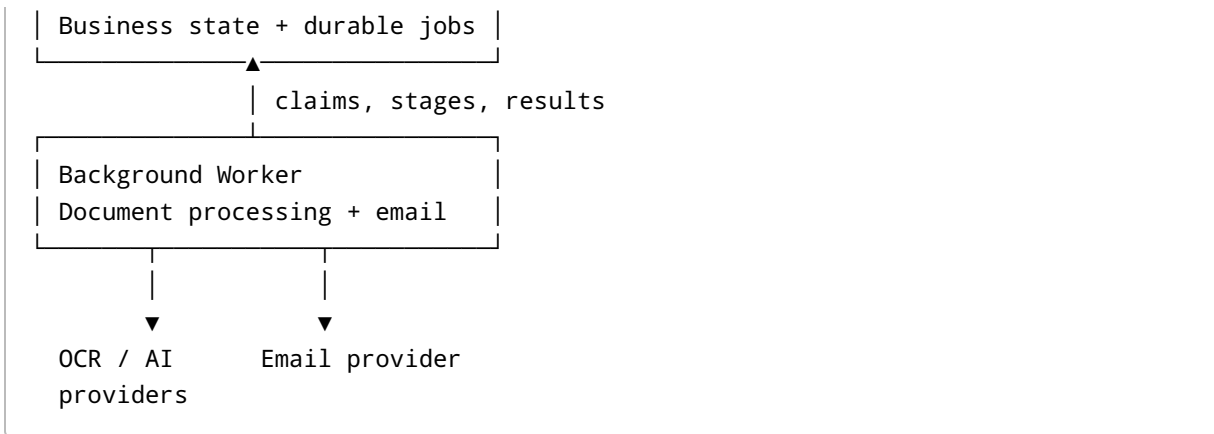
Complexity intentionally avoided

Release 1 does not introduce:

- event sourcing
- CQRS
- GraphQL
- generalized multi-tenancy
- arbitrary extraction schemas
- provider-routing tables
- Redis
- Kafka
- stage-per-message orchestration
- database-per-demo-session
- dynamic roles
- public developer APIs
- webhook infrastructure
- generalized workflow definitions

System diagram





Identifier and time conventions

Recommendation

Use UUID primary keys generated by the trusted backend or database for all business, job, artifact, audit, and public resource identities.

Alternatives

- Sequential integers are smaller and faster but should not be publicly exposed.
- Separate integer primary keys plus public UUIDs create two identifiers for every record.
- Sortable UUID variants are attractive but add runtime or database-version dependencies.

Decision

Use one UUID identifier per record in Release 1. Do not expose sequential identifiers.

Time

- Store all timestamps as `timestampz`.
- Treat database and backend time as authoritative.
- Store and compare time in UTC.
- Convert to the user's locale only in the frontend.
- Date-only policy fields use PostgreSQL `date`, not timestamps.

Concurrency

Mutable aggregate records use:

- `row_version bigint NOT NULL DEFAULT 1`
- incremented for every accepted mutation
- exposed to clients as an opaque string such as `rv:17`

Correlation and idempotency

- Correlation identifiers use UUID.
 - Idempotency keys are caller-generated strings up to 128 characters.
 - Idempotency keys are scoped by actor, route, and environment.
-

2. Goals and non-goals

2.1 Goals

- Exact module ownership
- Referential integrity
- Separation of source, candidate, review, and approval
- Immutable candidate and approved history
- Safe editable review state
- Safe repeated HTTP actions
- Stale-data detection
- Efficient Overview, Customers, Documents, Review Queue, and Audit queries
- Durable recoverable jobs
- Provider-neutral business tables
- Per-session demo isolation
- Clear frontend contracts
- Explainable transaction boundaries
- Predictable migrations
- Contract-test readiness
- Interview-level explainability

2.2 Non-goals

- General CRM schema
 - General multi-tenant SaaS platform
 - Event sourcing
 - CQRS
 - Graph database
 - Document database as primary storage
 - Runtime provider routing
 - Arbitrary extraction schemas
 - General workflow engine
 - Public customer APIs
 - Voice AI
 - Telephony
 - Billing or subscriptions
 - Custom permission matrices
 - Public webhook platform
-

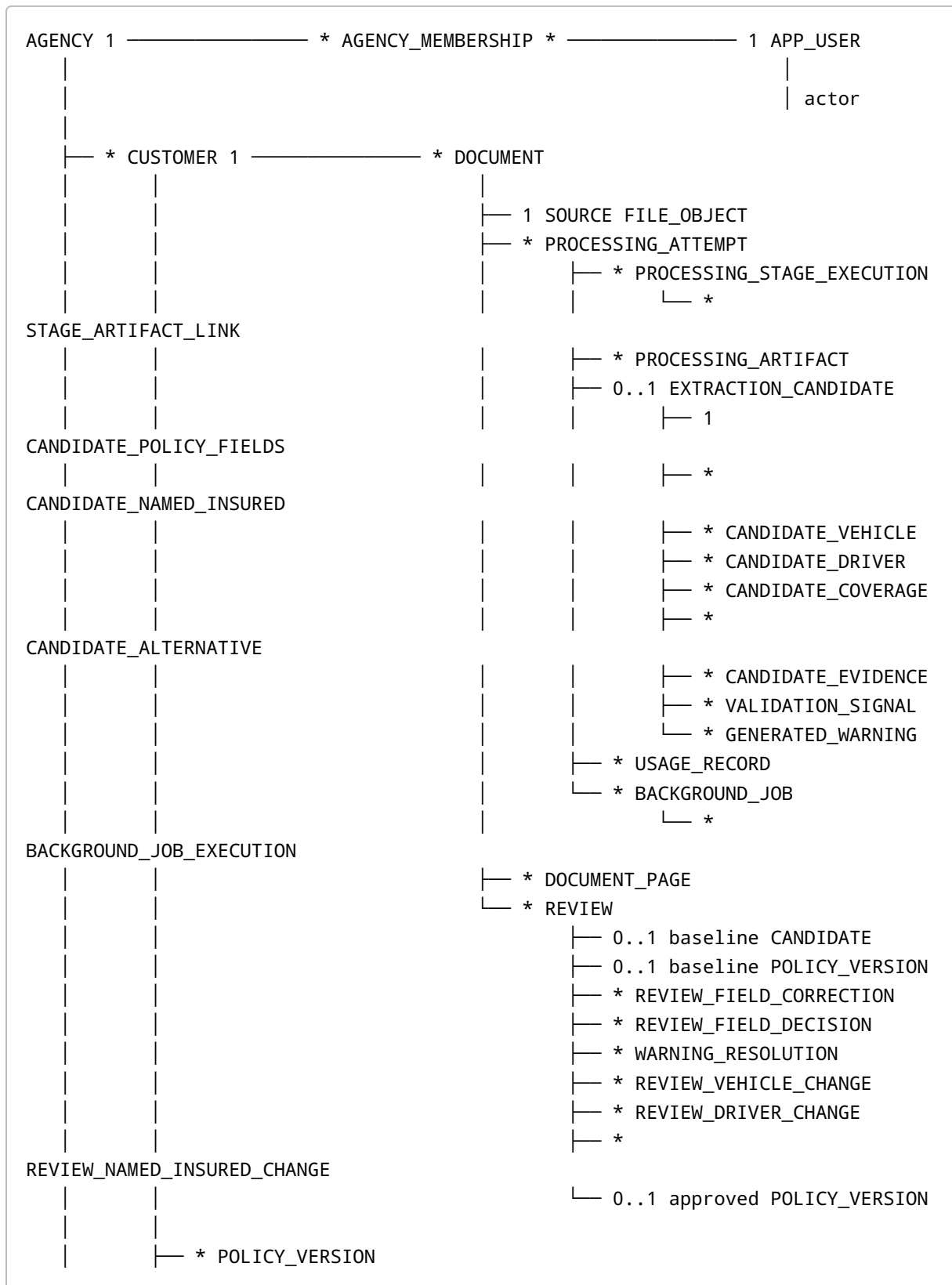
3. Source-of-truth and ownership map

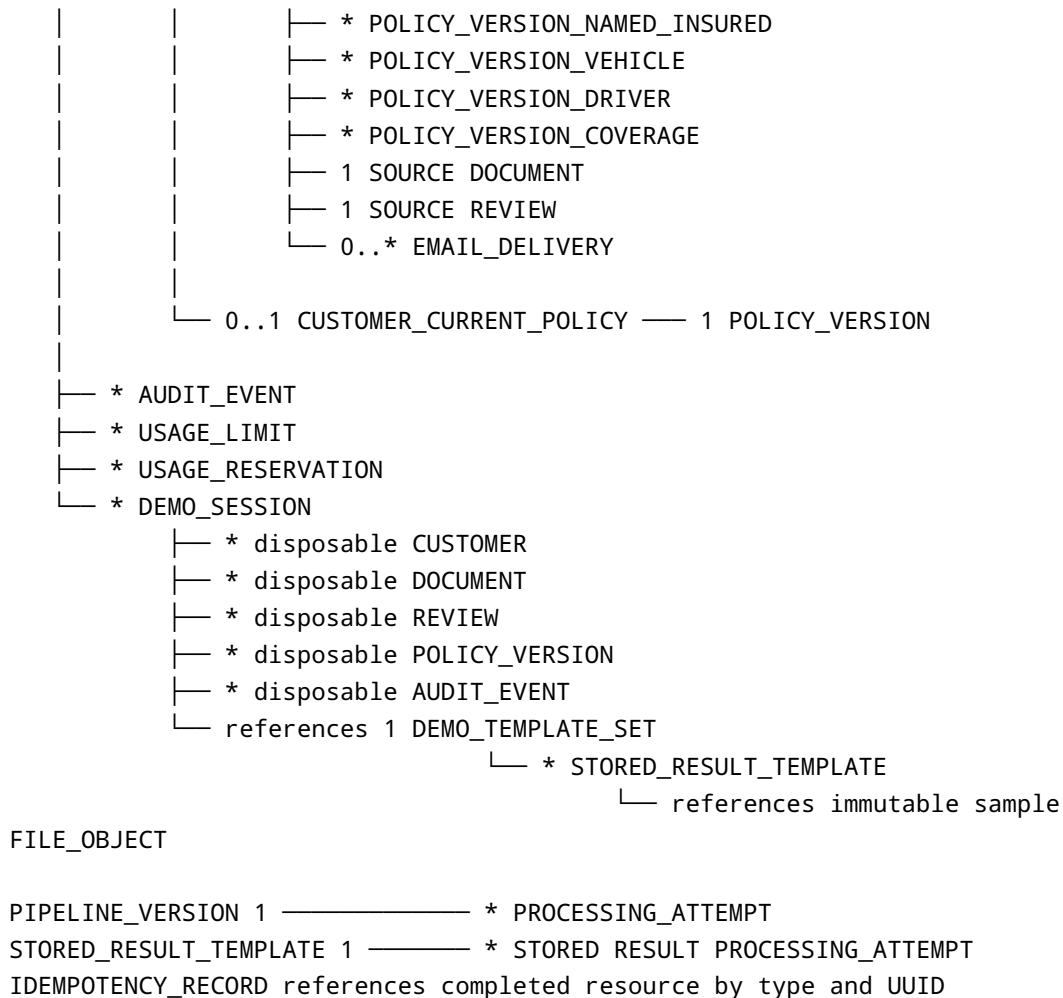
Concept	Owning module	Authoritative record	State	Creator	Updater	Main
Staff identity	Identity	<code>app_users</code>	Mutable profile, stable auth subject	Backend after verified sign-in	Backend	All m
Agency context	Identity	<code>agencies</code>	Mutable metadata	Seed/ migration	Owner operation	All m
Staff membership	Identity	<code>agency_memberships</code>	Mutable	Backend/ seed	Backend	Auth
Demo session	Demo	<code>demo_sessions</code>	Mutable until expired	Demo API	Demo API/ cleanup	Dem mod
Customer	Customers	<code>customers</code>	Mutable	Staff/demo user	Staff/demo user	Docu review
File object	Documents/ Platform	<code>file_objects</code>	Metadata mutable until deletion	Upload or worker	Storage service routines	Docu proc
Document	Documents	<code>documents</code>	Mutable lifecycle	Upload completion	Backend state transitions	All d work
Upload authorization	Documents	<code>upload_authorizations</code>	Mutable until terminal	Upload API	Upload API/ cleanup	Uplo
Processing attempt	Processing	<code>processing_attempts</code>	Mutable until terminal, immutable thereafter	Processing command	Worker/ backend	Docu review
Background job	Processing	<code>background_jobs</code>	Mutable lifecycle	Backend	Worker/ schedulers	Work oper
Job execution	Processing	<code>background_job_executions</code>	Immutable	Worker	None after completion	Diag

Concept	Owning module	Authoritative record	State	Creator	Updater	Main
Stage execution	Processing	<code>processing_stage_executions</code>	Immutable after terminal	Worker	Worker while active	UX p evalu
Parser/OCR/ AI artifact	Processing	<code>processing_artifacts</code>	Immutable content metadata	Worker	Cleanup state only	Pipel evalu
PDF page	Processing	<code>document_pages</code>	Immutable after inspection completion	Worker	Worker before attempt completion	Evide view
Candidate	Extraction	<code>extraction_candidates</code> and child tables	Immutable	Worker	Never	Revie evalu
Evidence	Extraction	<code>candidate_evidence</code>	Immutable	Worker	Never	Revie
Validation signal	Extraction	<code>validation_signals</code>	Immutable	Worker	Never	Warn gene evalu
Generated warning	Extraction	<code>generated_warnings</code>	Immutable	Worker	Never	Revie
Review	Review	<code>reviews</code>	Mutable until terminal	Backend	Review actions	UX/a
Correction	Review	<code>review_field_corrections</code>	Current row mutable through supersession; history preserved	Reviewer	Reviewer	Read appr
Field decision	Review	<code>review_field_decisions</code>	Mutable through supersession	Reviewer	Reviewer	Read
Warning resolution	Review	<code>warning_resolutions</code>	Mutable until review terminal	Reviewer	Reviewer	Read appr
Added/ removed item	Review	Review item-change tables	Mutable while review open	Reviewer	Reviewer	Appr

Concept	Owning module	Authoritative record	State	Creator	Updater	Main
Approved policy version	Policy Versions	<code>policy_versions</code> and children	Immutable	Approval transaction	Never	Custo polic
Current-policy selection	Policy Versions	<code>customer_current_policies</code>	Mutable pointer	Approval transaction	Approval transaction	Custo
Audit event	Audit	<code>audit_events</code>	Append-only	Backend/ worker	Never	Activ
Usage allowance	Usage	<code>usage_limits</code>	Mutable owner configuration	Seed/ owner operation	Owner operation	Back
Usage reservation	Usage	<code>usage_reservations</code>	Mutable lifecycle	Processing command	Worker/ cleanup	Limit
Usage record	Usage	<code>usage_records</code>	Immutable	Worker/ backend	Never	Usag
Email delivery	Email	<code>email_deliveries</code>	Mutable delivery lifecycle	Backend	Worker	Appr Sum
Stored Result template	Demo	<code>stored_result_templates</code>	Immutable version	Controlled release process	Never	Dem
Demo template set	Demo	<code>demo_template_sets</code>	Immutable version	Controlled release process	Never	Dem reset
Pipeline configuration	Processing	<code>pipeline_versions</code>	Immutable once approved	Controlled release process	Status only	Atter jobs
Idempotency result	Platform	<code>idempotency_records</code>	Mutable until terminal	FastAPI	FastAPI	Repe sens route
Benchmark source/result	Evaluation	Version-controlled files and private artifacts	Immutable versions	Evaluation process	New versions only	Own

4. Domain relationship diagram





- A document owns its attempts.
- An attempt owns its stage executions, artifacts, and candidate.
- A candidate owns its candidate entities, evidence, validation signals, and warnings.
- A review references a candidate or approved version but does not own or mutate its baseline.
- A policy version owns its approved child records.
- Audit events reference business records but do not own them.
- Stored Result templates are shared immutable references; replay creates new disposable records.

5.1 Storage recommendation

Use `text` columns with explicit database `CHECK` constraints rather than PostgreSQL enums.

Why

- Easier forward-only migrations
- Easier temporary compatibility between web and worker versions
- No type-level enum migration locking concerns
- Values remain explicit and constrained

Trade-off

The same value lists must be maintained carefully in database constraints, backend contracts, and tests.

Reference-data tables are reserved for owner-editable business configuration, not fixed lifecycle values.

5.2 Document disposition

Stored in `documents.disposition`.

Allowed values:

- `PROCESSING`
- `NEEDS_REVIEW`
- `APPROVED`
- `REJECTED`
- `FAILED`

Transitions:

- New accepted document → `PROCESSING`
- Successful first attempt → `NEEDS_REVIEW`
- Approval → `APPROVED`
- Rejection → `REJECTED`
- Failed attempt without prior approval → `FAILED`
- Approved document remains `APPROVED` during later reprocessing

Composite states are derived:

- Approved · Reprocessing
- Approved · New Review Available
- Approved · Reprocessing Failed

Historical completed values change only through valid lifecycle transitions.

5.3 Attempt origin

Stored:

- `INITIAL`

- INTERNAL_RECOVERY
- TECHNICAL_RETRY
- REPROCESSING
- STORED_RESULT

Immutable after attempt creation.

5.4 Attempt status

Stored:

- PENDING
- CLAIMED
- RUNNING
- SUCCEEDED
- FAILED
- CANCELLED
- SUPERSEDED

Allowed high-level transitions:

```
PENDING → CLAIMED → RUNNING → SUCCEEDED
PENDING → CANCELLED
CLAIMED → PENDING      lease expiry recovery
CLAIMED → RUNNING
RUNNING → FAILED
RUNNING → PENDING      safe reclaim only before an irreversible stage
PENDING/RUNNING → SUPERSEDED only by explicit replacement logic
```

Terminal statuses are immutable.

5.5 Processing stages

Stored:

- ACCEPTED
- INSPECTING
- EXTRACTING_DIRECT_TEXT
- EVALUATING_TEXT_QUALITY
- PREPARING_OCR
- RUNNING_OCR
- CLASSIFYING_DOCUMENT
- CHECKING_POLICY_COUNT
- PREPARING_CONTEXT
- EXTRACTING_FIELDS
- VALIDATING_STRUCTURE

- NORMALIZING
- VALIDATING_BUSINESS_RULES
- PREPARING_EVIDENCE
- GENERATING_WARNINGS
- PUBLISHING_CANDIDATE
- FINALIZING_USAGE
- COMPLETED

Stored stage executions never change stage identity.

5.6 Stage execution status

- PENDING
- RUNNING
- SUCCEEDED
- FAILED
- SKIPPED
- REUSED

Terminal after completion.

5.7 Review origin

- CANDIDATE
- CORRECTION

Immutable.

5.8 Review status

- OPEN
- READY
- APPROVED
- REJECTED
- STALE
- ABANDONED

READY is a cached server state, never accepted from the frontend.

Final statuses are immutable.

5.9 Warning severity

- INFORMATION
- IMPORTANT
- CRITICAL

Generated warning severity is immutable.

5.10 Warning resolution

- UNRESOLVED
- CORRECTED
- ACKNOWLEDGED
- MARKED_NOT_PRESENT
- CONFIRMED_DIFFERENCE
- REMOVED_BY_REVIEW_CHANGE

NOT_APPLICABLE is omitted because an irrelevant generated warning should be corrected through a new pipeline version rather than hidden casually.

5.11 Policy designation

Do not store CURRENT or HISTORICAL on immutable policy versions.

Derive:

- Current when referenced by customer_current_policies
- Historical otherwise

This preserves policy-version immutability.

5.12 Evidence source

- DIRECT_TEXT
- OCR_TEXT
- PAGE_IMAGE
- COMBINED
- PAGE_ONLY

Immutable.

5.13 Evidence verification

- VERIFIED
- PARTIALLY_VERIFIED
- PAGE_ONLY
- UNVERIFIED

5.14 Artifact category

- ORIGINAL_PDF
- DIRECT_TEXT
- OCR_TEXT
- PAGE_IMAGE
- RAW_OCR_RESPONSE
- RAW_AI_RESPONSE

- PREPARED_CONTEXT
- CANDIDATE_SNAPSHOT
- VALIDATION_OUTPUT
- EVIDENCE_OUTPUT
- STORED_RESULT_SOURCE
- BENCHMARK_ARTIFACT

5.15 Artifact retention category

- BUSINESS
- EVALUATION
- DIAGNOSTIC
- TEMPORARY
- DEMO_TEMPLATE

5.16 Job status

- AVAILABLE
- CLAIMED
- RUNNING
- SUCCEEDED
- FAILED
- CANCELLED
- DEAD_LETTER

5.17 Job type

- PROCESS_DOCUMENT_ATTEMPT
- REPLAY_STORED_RESULT
- SEND_ACKNOWLEDGEMENT_EMAIL
- EXPIRE_DEMO_SESSION
- RESET_DEMO_SESSION
- CLEANUP_ARTIFACT
- CLEANUP_UPLOAD
- DETECT_STALE_ATTEMPT
- RELEASE_USAGE_RESERVATION

5.18 Usage category

- DOCUMENT_SUBMISSION
- PAGE_PROCESSED
- OCR_PAGE
- AI_INPUT
- AI_OUTPUT
- AI_ATTEMPT
- STRUCTURAL_REPAIR
- INTERNAL_RECOVERY

- TECHNICAL_RETRY
- REPROCESSING
- FAILED_ATTEMPT
- STORED_RESULT
- EMAIL_ATTEMPT

5.19 Usage mode

- LIVE
- STORED
- SIMULATED

5.20 Email status

- PENDING
- SENDING
- SENT
- FAILED
- SIMULATED

5.21 Demo session status

- ACTIVE
- RESETTING
- EXPIRED
- CLEANUP_PENDING
- CLEANED
- FAILED

6. Identity and agency-context tables

6.1 agencies

Purpose: One business context per environment.

Column	Type	Null	Default	Rules
id	uuid	No	generated	Primary key
name	text	No	—	Length 1-160
slug	text	No	—	Unique, lowercase format
environment_kind	text	No	—	DEVELOPMENT, DEMO, or PRODUCTION
created_at	timestampz	No	now()	Immutable

Column	Type	Null	Default	Rules
<code>updated_at</code>	<code>timestampz</code>	No	<code>now()</code>	Server updated
<code>row_version</code>	<code>bigint</code>	No	<code>1</code>	Positive
<code>archived_at</code>	<code>timestampz</code>	Yes	<code>NULL</code>	Internal only

Indexes:

- Unique `slug`
- Index `environment_kind`

Mutability:

- Name and archived state mutable
- Identity and environment kind immutable after use

Retention:

- Permanent per environment

6.2 `app_users`

Purpose: Connect a verified Supabase Auth subject to an application actor.

Column	Type	Null	Default	Rules
<code>id</code>	<code>uuid</code>	No	generated	Primary key
<code>auth_subject</code>	<code>uuid</code>	No	—	Unique Supabase user identity
<code>display_name</code>	<code>text</code>	No	—	Length 1–160
<code>email_snapshot</code>	<code>text</code>	Yes	<code>NULL</code>	Non-authoritative display value
<code>status</code>	<code>text</code>	No	<code>ACTIVE</code>	<code>ACTIVE</code> , <code>DISABLED</code>
<code>created_at</code>	<code>timestampz</code>	No	<code>now()</code>	—
<code>updated_at</code>	<code>timestampz</code>	No	<code>now()</code>	—
<code>row_version</code>	<code>bigint</code>	No	<code>1</code>	Positive
<code>disabled_at</code>	<code>timestampz</code>	Yes	<code>NULL</code>	Required if disabled

Constraints:

- Unique `auth_subject`
- Disabled timestamp consistency

6.3 agency_memberships

Purpose: Backend authorization for the single agency context.

Column	Type	Null	Default	Rules
id	uuid	No	generated	Primary key
agency_id	uuid	No	—	FK agencies
app_user_id	uuid	No	—	FK app_users
status	text	No	ACTIVE	ACTIVE, INACTIVE
created_at	timestampz	No	now()	—
updated_at	timestampz	No	now()	—
row_version	bigint	No	1	Positive
deactivated_at	timestampz	Yes	NULL	—

Constraints:

- Unique (agency_id, app_user_id)
- No roles column in Release 1

Why Auth identity alone is insufficient

Supabase Auth proves who the caller is. It does not prove:

- agency membership
- demo versus staff context
- object ownership
- whether access has been disabled
- whether the actor can use a specific demo session

FastAPI resolves these business facts.

7. Customer tables

7.1 Address recommendation

Use structured address columns directly on customers.

Simpler alternative

One free-text address field.

Why structured columns are justified

The approved UX compares:

- customer contact address
- policy mailing address

Structured fields support:

- readable display
- state and ZIP comparisons
- explicit contact-address updates
- search and validation

A separate address table is unnecessary because Release 1 stores only one customer contact address.

7.2 customers

Column	Type	Null	Default	Rules
<code>id</code>	<code>uuid</code>	No	generated	Primary key
<code>agency_id</code>	<code>uuid</code>	No	—	FK <code>agencies</code>
<code>demo_session_id</code>	<code>uuid</code>	Yes	<code>NULL</code>	FK <code>demo_sessions</code>
<code>full_name</code>	<code>text</code>	No	—	Length 1–200
<code>normalized_name</code>	<code>text</code>	No	—	Search-only canonical form
<code>email</code>	<code>text</code>	Yes	<code>NULL</code>	Validated by backend
<code>normalized_email</code>	<code>text</code>	Yes	<code>NULL</code>	Lowercase search value
<code>phone</code>	<code>text</code>	Yes	<code>NULL</code>	Display form
<code>normalized_phone</code>	<code>text</code>	Yes	<code>NULL</code>	Digits/E.164-like search form
<code>address_line1</code>	<code>text</code>	Yes	<code>NULL</code>	—
<code>address_line2</code>	<code>text</code>	Yes	<code>NULL</code>	—
<code>city</code>	<code>text</code>	Yes	<code>NULL</code>	—
<code>state_code</code>	<code>text</code>	Yes	<code>NULL</code>	Two-character U.S. code when supplied
<code>postal_code</code>	<code>text</code>	Yes	<code>NULL</code>	Preserve leading zeros
<code>country_code</code>	<code>text</code>	No	<code>US</code>	Two characters
<code>search_text</code>	<code>text</code>	No	—	Backend-maintained combined search value

Column	Type	Null	Default	Rules
<code>created_by</code>	<code>uuid</code>	No	—	FK <code>app_users</code>
<code>created_at</code>	<code>timestampz</code>	No	<code>now()</code>	—
<code>updated_at</code>	<code>timestampz</code>	No	<code>now()</code>	—
<code>row_version</code>	<code>bigint</code>	No	1	Positive
<code>archived_at</code>	<code>timestampz</code>	Yes	NULL	Internal only

Constraints:

- If `demo_session_id` is set, it must belong to the same agency; backend transaction enforces cross-table match.
- Email and phone are optional.
- At least full name is required.
- No uniqueness constraint on name, email, or phone because legitimate duplicates exist.

Duplicate indicators are calculated by backend queries. They are not permanent truth.

8. Document and storage-reference tables

8.1 Separate file objects from documents

Recommendation: Use a separate `file_objects` table.

A document is a business record. A file object is a private storage object.

This separation supports:

- upload authorization before document creation
- source files
- page images
- raw artifacts
- synthetic shared files
- cleanup without deleting business records incorrectly

8.2 `file_objects`

Column	Type	Null	Default	Rules
<code>id</code>	<code>uuid</code>	No	generated	Primary key
<code>agency_id</code>	<code>uuid</code>	No	—	FK <code>agencies</code>
<code>demo_session_id</code>	<code>uuid</code>	Yes	NULL	FK <code>demo_sessions</code>

Column	Type	Null	Default	Rules
<code>storage_bucket</code>	<code>text</code>	No	—	Approved bucket key
<code>storage_key</code>	<code>text</code>	No	—	Server-generated
<code>purpose</code>	<code>text</code>	No	—	<code>SOURCE</code> , <code>ARTIFACT</code> , <code>TEMPLATE</code> , <code>BENCHMARK</code>
<code>original_filename</code>	<code>text</code>	Yes	<code>NULL</code>	Display only
<code>detected_mime_type</code>	<code>text</code>	No	—	Never trusted from browser alone
<code>byte_size</code>	<code>bigint</code>	No	—	Nonnegative
<code>content_sha256</code>	<code>text</code>	No	—	64 lowercase hexadecimal characters
<code>storage_status</code>	<code>text</code>	No	<code>AVAILABLE</code>	<code>PENDING</code> , <code>AVAILABLE</code> , <code>DELETION_PENDING</code> , <code>DELETED</code> , <code>MISSING</code>
<code>restricted_access</code>	<code>boolean</code>	No	<code>true</code>	Must be true for source and raw artifacts
<code>created_by</code>	<code>uuid</code>	Yes	<code>NULL</code>	FK <code>app_users</code> ; system-created allowed
<code>created_at</code>	<code>timestampz</code>	No	<code>now()</code>	—
<code>deleted_at</code>	<code>timestampz</code>	Yes	<code>NULL</code>	—
<code>retention_category</code>	<code>text</code>	No	—	Controlled value
<code>expires_at</code>	<code>timestampz</code>	Yes	<code>NULL</code>	Required for temporary categories
<code>cleanup_status</code>	<code>text</code>	No	<code>NOT_REQUIRED</code>	<code>NOT_REQUIRED</code> , <code>PENDING</code> , <code>RUNNING</code> , <code>FAILED</code> , <code>COMPLETED</code>
<code>cleanup_attempts</code>	<code>integer</code>	No	<code>0</code>	Nonnegative
<code>last_cleanup_error_ref</code>	<code>text</code>	Yes	<code>NULL</code>	Diagnostic reference only

Constraints:

- Unique (storage_bucket, storage_key)
- byte_size >= 0
- Source PDFs must use private bucket policy.
- Temporary retention requires expiry.

8.3 documents

Column	Type	Null	Default	Rules
id	uuid	No	generated	Primary key
agency_id	uuid	No	—	FK
demo_session_id	uuid	Yes	NULL	FK
customer_id	uuid	No	—	FK customers
source_file_id	uuid	No	—	Unique FK file_objects
display_name	text	No	—	Sanitized filename
page_count	smallint	No	—	1-10
disposition	text	No	PROCESSING	Controlled value
duplicate_of_document_id	uuid	Yes	NULL	Self FK
active_attempt_id	uuid	Yes	NULL	FK processing_attempts ; transaction-maintained
current_reviewable_attempt_id	uuid	Yes	NULL	FK processing_attempts
uploaded_by	uuid	No	—	FK app_users
uploaded_at	timestampz	No	now()	—
last_activity_at	timestampz	No	now()	Operational sorting
created_at	timestampz	No	now()	—
updated_at	timestampz	No	now()	—
row_version	bigint	No	1	Positive
archived_at	timestampz	Yes	NULL	Internal only

Constraints:

- Unique source_file_id

- `page_count BETWEEN 1 AND 10`
- Source file must have purpose `SOURCE`, enforced by backend transaction.
- Active and reviewable attempts must belong to the same document, enforced transactionally.
- Demo customer and document must share the same session.

9. Upload authorization and completion records

9.1 Dedicated record

Recommendation: Persist upload authorization.

It protects:

- allowed customer
- generated storage path
- expiry
- expected limits
- repeated completion
- orphan cleanup
- direct-to-storage security

9.2 `upload_authorizations`

Column	Type	Null	Default
<code>id</code>	<code>uuid</code>	No	generated
<code>agency_id</code>	<code>uuid</code>	No	—
<code>demo_session_id</code>	<code>uuid</code>	Yes	<code>NULL</code>
<code>customer_id</code>	<code>uuid</code>	No	—
<code>requested_by</code>	<code>uuid</code>	No	—
<code>replacement_of_document_id</code>	<code>uuid</code>	Yes	<code>NULL</code>
<code>storage_bucket</code>	<code>text</code>	No	—
<code>storage_key</code>	<code>text</code>	No	server-generated
<code>expected_filename</code>	<code>text</code>	No	—
<code>expected_mime_type</code>	<code>text</code>	No	<code>application/pdf</code>
<code>maximum_bytes</code>	<code>bigint</code>	No	<code>15728640</code>
<code>maximum_pages</code>	<code>smallint</code>	No	<code>10</code>
<code>status</code>	<code>text</code>	No	<code>AUTHORIZED</code>

Column	Type	Null	Default
<code>expires_at</code>	<code>timestamptz</code>	No	—
<code>completed_at</code>	<code>timestamptz</code>	Yes	<code>NULL</code>
<code>confirmed_file_id</code>	<code>uuid</code>	Yes	<code>NULL</code>
<code>created_document_id</code>	<code>uuid</code>	Yes	<code>NULL</code>
<code>idempotency_record_id</code>	<code>uuid</code>	No	—
<code>rejection_code</code>	<code>text</code>	Yes	<code>NULL</code>
<code>created_at</code>	<code>timestamptz</code>	No	<code>now()</code>
<code>updated_at</code>	<code>timestamptz</code>	No	<code>now()</code>
<code>row_version</code>	<code>bigint</code>	No	<code>1</code>

Statuses:

- `AUTHORIZED`
- `UPLOADED`
- `COMPLETED`
- `REJECTED`
- `EXPIRED`
- `CANCELLED`

Constraints:

- Unique (`storage_bucket`, `storage_key`)
- Maximum values fixed to approved limits
- Demo sessions cannot request arbitrary local uploads
- `created_document_id` unique when set

Completion behavior

A repeated completion request with the same idempotency key returns the existing document.

A completion request with a changed object identity returns an idempotency conflict.

Expired incomplete objects are cleaned by `CLEANUP_UPLOAD`.

10. Processing-attempt tables

10.1 `processing_attempts`

Column	Type	Null	Default	Purpose
<code>id</code>	<code>uuid</code>	No	generated	Primary key
<code>document_id</code>	<code>uuid</code>	No	—	FK
<code>origin</code>	<code>text</code>	No	—	Controlled
<code>parent_attempt_id</code>	<code>uuid</code>	Yes	<code>NULL</code>	Previous attempt
<code>triggered_by</code>	<code>uuid</code>	Yes	<code>NULL</code>	User or system
<code>pipeline_version_id</code>	<code>uuid</code>	No	—	FK
<code>status</code>	<code>text</code>	No	<code>PENDING</code>	Controlled
<code>active_stage</code>	<code>text</code>	No	<code>ACCEPTED</code>	Controlled
<code>processing_mode</code>	<code>text</code>	No	<code>LIVE</code>	<code>LIVE</code> or <code>STORED</code>
<code>stored_result_template_id</code>	<code>uuid</code>	Yes	<code>NULL</code>	Required for Stored Result
<code>technical_retry_ordinal</code>	<code>smallint</code>	Yes	<code>NULL</code>	1 or 2
<code>internal_recovery_ordinal</code>	<code>smallint</code>	Yes	<code>NULL</code>	Positive
<code>started_at</code>	<code>timestampz</code>	Yes	<code>NULL</code>	—
<code>completed_at</code>	<code>timestampz</code>	Yes	<code>NULL</code>	—
<code>last_progress_at</code>	<code>timestampz</code>	No	<code>now()</code>	Stale detection
<code>failure_category</code>	<code>text</code>	Yes	<code>NULL</code>	Stable taxonomy
<code>failure_retryable</code>	<code>boolean</code>	Yes	<code>NULL</code>	—
<code>replacement_required</code>	<code>boolean</code>	No	<code>false</code>	—
<code>paid_usage_possible</code>	<code>boolean</code>	No	<code>false</code>	User error contract
<code>diagnostic_reference</code>	<code>text</code>	No	generated	Unique
<code>correlation_id</code>	<code>uuid</code>	No	generated	Trace
<code>reused_artifacts_summary</code>	<code>jsonb</code>	No	<code>{}</code>	Diagnostic, non-authoritative
<code>row_version</code>	<code>bigint</code>	No	<code>1</code>	Concurrency
<code>created_at</code>	<code>timestampz</code>	No	<code>now()</code>	—

Column	Type	Null	Default	Purpose
<code>updated_at</code>	<code>timestampz</code>	No	<code>now()</code>	—

Constraints:

- Partial unique index on `document_id` where status is `PENDING`, `CLAIMED`, or `RUNNING`
- `TECHNICAL_RETRY` requires ordinal 1 or 2
- Unique `(document_id, technical_retry_ordinal)` where origin is Technical Retry
- `STORED_RESULT` requires template and `processing_mode = STORED`
- Non-Stored Result cannot reference a Stored Result template
- Terminal status requires `completed_at`
- Completed attempts cannot change pipeline identity or origin
- Parent attempt must belong to the same document, enforced transactionally

Active versus current reviewable

- `documents.active_attempt_id` identifies current live work.
- `documents.current_reviewable_attempt_id` identifies the candidate currently presented for review.
- They may be different.
- An approved document may have both an approved policy and a new active or reviewable attempt.

11. Background-job tables

11.1 Queue strategy comparison

A. One orchestration job per processing attempt

Advantages:

- simplest queue
- natural attempt ownership
- fewer job rows
- easier stage reuse
- easy worker restart through durable stage records

Risk:

- one job may run for several minutes

B. One job per stage

Advantages:

- fine-grained retries

- independent stage scaling

Problems:

- many state transitions
- orchestration complexity
- duplicate stage scheduling
- more queue and compatibility burden

C. Hybrid

One orchestration job plus separate email and cleanup jobs.

Recommendation

Use Option C.

One document-processing job advances the pipeline. It commits after every stage so the job can recover safely. Email and maintenance remain separate jobs.

11.2 background_jobs

Column	Type	Null	Default
id	uuid	No	generated
job_type	text	No	—
status	text	No	AVAILABLE
related_attempt_id	uuid	Yes	NULL
related_email_delivery_id	uuid	Yes	NULL
related_demo_session_id	uuid	Yes	NULL
related_file_object_id	uuid	Yes	NULL
priority	smallint	No	100
available_at	timestampz	No	now()
claim_owner	text	Yes	NULL
claimed_at	timestampz	Yes	NULL
lease_expires_at	timestampz	Yes	NULL
heartbeat_at	timestampz	Yes	NULL
execution_count	integer	No	0

Column	Type	Null	Default
maximum_executions	integer	No	—
payload_version	integer	No	1
handler_version_required	text	No	—
payload	jsonb	No	{}
idempotency_identity	text	No	—
last_failure_category	text	Yes	NULL
last_diagnostic_reference	text	Yes	NULL
completed_at	timestampz	Yes	NULL
created_at	timestampz	No	now()
updated_at	timestampz	No	now()
row_version	bigint	No	1

JSONB justification:

- Job payloads vary by job type.
- Payload contains versioned resource identifiers and options, not business truth or provider secrets.
- Validation is performed according to `job_type` and `payload_version`.
- No JSON index is required.

Constraints:

- Unique `(job_type, idempotency_identity)`
- At least one related aggregate appropriate to job type
- Claim columns must be all null or all populated consistently
- `execution_count >= 0`
- `maximum_executions > 0`
- Terminal status requires completion time

11.3 background_job_executions

Column	Type	Null	Default
id	uuid	No	generated
job_id	uuid	No	FK
execution_number	integer	No	—
worker_identity	text	No	—

Column	Type	Null	Default
<code>started_at</code>	<code>timestampz</code>	No	<code>now()</code>
<code>last_heartbeat_at</code>	<code>timestampz</code>	Yes	<code>NULL</code>
<code>ended_at</code>	<code>timestampz</code>	Yes	<code>NULL</code>
<code>outcome</code>	<code>text</code>	Yes	<code>NULL</code>
<code>failure_category</code>	<code>text</code>	Yes	<code>NULL</code>
<code>diagnostic_reference</code>	<code>text</code>	No	<code>generated</code>
<code>metrics</code>	<code>jsonb</code>	No	<code>{}</code>

Constraints:

- Unique `(job_id, execution_number)`
- Immutable after `ended_at`

12. Worker-lease and claim contract

Claim selection

A worker selects jobs that:

- have status `AVAILABLE`
- have `available_at <= now()`
- have compatible handler version
- have not exceeded maximum executions
- are ordered by priority, then availability, then creation time

Claim outcome

Exactly one worker changes the job to `CLAIMED`, writes:

- claim owner
- claim time
- lease expiry
- heartbeat
- incremented execution count

Concurrent workers must observe either:

- successful ownership
- no claim

They must never both receive a valid lease.

Lease duration

Lease duration must exceed normal heartbeat intervals but remain short enough for recovery.

The exact duration is deployment configuration, not a business constant.

Heartbeat

A claimed or running job extends its lease periodically while active.

Claim expiry

An expired claim may be reclaimed only when:

- current lease is in the past
- job is not terminal
- attempt remains eligible
- previous stage commits are visible

Safe reclaim

The new worker:

1. Validates the attempt.
2. Reads completed stage history.
3. Reuses valid artifacts.
4. Restarts only the incomplete stage.
5. Never repeats an irreversible completed transaction.

Graceful shutdown

A worker should:

- stop claiming new jobs
- finish the current safe stage where possible
- release or allow expiry of the lease
- never mark success before committing stage output

Poisoned jobs

A job becomes `DEAD_LETTER` when:

- maximum internal executions are exhausted
- failure is non-retryable
- handler compatibility cannot be satisfied within operational recovery

The related business attempt becomes Failed through a separate terminal transaction.

13. Processing-stage records

Recommendation

Use:

- one current `active_stage` on `processing_attempts`
- append-preserved `processing_stage_executions`
- explicit artifact links

Audit events alone are too coarse. Job records alone cannot represent repeated and reused stages.

13.1 `processing_stage_executions`

Column	Type	Null	Default
<code>id</code>	<code>uuid</code>	No	generated
<code>attempt_id</code>	<code>uuid</code>	No	FK
<code>stage</code>	<code>text</code>	No	—
<code>execution_number</code>	<code>smallint</code>	No	1
<code>status</code>	<code>text</code>	No	PENDING
<code>configuration_identity</code>	<code>text</code>	No	—
<code>started_at</code>	<code>timestampz</code>	Yes	NULL
<code>completed_at</code>	<code>timestampz</code>	Yes	NULL
<code>duration_ms</code>	<code>bigint</code>	Yes	NULL
<code>reused_from_stage_execution_id</code>	<code>uuid</code>	Yes	NULL
<code>usage_record_id</code>	<code>uuid</code>	Yes	NULL
<code>failure_category</code>	<code>text</code>	Yes	NULL
<code>diagnostic_reference</code>	<code>text</code>	No	generated
<code>metrics</code>	<code>jsonb</code>	No	{}
<code>created_at</code>	<code>timestampz</code>	No	now()

Constraints:

- Unique (`attempt_id`, `stage`, `execution_number`)

- Reused status requires source execution
- Completed or failed stage requires completion time
- Immutable after terminal

13.2 stage_artifact_links

Column	Type	Null	Rules
id	uuid	No	PK
stage_execution_id	uuid	No	FK
artifact_id	uuid	No	FK
direction	text	No	INPUT or OUTPUT
created_at	timestampz	No	—

Unique (stage_execution_id, artifact_id, direction).

14. Artifact metadata tables

14.1 processing_artifacts

Column	Type	Null	Default
id	uuid	No	generated
attempt_id	uuid	Yes	NULL
category	text	No	—
file_object_id	uuid	Yes	NULL
inline_text	text	Yes	NULL
inline_json	jsonb	Yes	NULL
content_type	text	No	—
byte_size	bigint	No	0
content_sha256	text	No	—
produced_by_stage_execution_id	uuid	Yes	NULL
configuration_identity	text	No	—
page_start	smallint	Yes	NULL
page_end	smallint	Yes	NULL

Column	Type	Null	Default
<code>authoritative_level</code>	<code>text</code>	No	—
<code>restricted_access</code>	<code>boolean</code>	No	<code>true</code>
<code>retention_category</code>	<code>text</code>	No	—
<code>expires_at</code>	<code>timestampz</code>	Yes	<code>NULL</code>
<code>cleanup_status</code>	<code>text</code>	No	<code>NOT_REQUIRED</code>
<code>deleted_at</code>	<code>timestampz</code>	Yes	<code>NULL</code>
<code>created_at</code>	<code>timestampz</code>	No	<code>now()</code>

Content-location constraint:

Exactly one of:

- `file_object_id`
- `inline_text`
- `inline_json`

must contain content.

Storage recommendations

Artifact	Location
Original PDF	<code>file_objects</code> in private Storage
Direct text under practical size	PostgreSQL <code>inline_text</code>
Large direct text	Private Storage
OCR text	PostgreSQL or Storage based on size
Page image	Private Storage
Raw OCR response	Private Storage or JSONB when small
Raw AI response	Restricted Storage or JSONB when small
Prepared context	Restricted Storage or inline text
Candidate snapshot	JSONB artifact for reproducibility; relational candidate remains authoritative
Validation output	Relational signals plus optional JSON artifact
Evidence output	Relational evidence plus optional snapshot

Artifact	Location
Stored Result source	Immutable Storage/template JSON
Benchmark artifact	Private Storage outside normal production access

JSONB is not authoritative for approved business data.

15. Page and source-text structures

15.1 Hybrid recommendation

Use relational page metadata and artifact-backed text.

This supports page queries without placing every coordinate or long OCR response into core relational columns.

15.2 `document_pages`

Column	Type	Null	Default
<code>id</code>	<code>uuid</code>	No	<code>generated</code>
<code>document_id</code>	<code>uuid</code>	No	<code>—</code>
<code>page_number</code>	<code>smallint</code>	No	<code>—</code>
<code>width_points</code>	<code>numeric(10,2)</code>	No	<code>—</code>
<code>height_points</code>	<code>numeric(10,2)</code>	No	<code>—</code>
<code>detected_rotation_degrees</code>	<code>smallint</code>	No	<code>0</code>
<code>duplicate_of_page_id</code>	<code>uuid</code>	Yes	<code>NULL</code>
<code>direct_text_artifact_id</code>	<code>uuid</code>	Yes	<code>NULL</code>
<code>ocr_text_artifact_id</code>	<code>uuid</code>	Yes	<code>NULL</code>
<code>primary_text_artifact_id</code>	<code>uuid</code>	Yes	<code>NULL</code>
<code>page_image_artifact_id</code>	<code>uuid</code>	Yes	<code>NULL</code>
<code>primary_source</code>	<code>text</code>	Yes	<code>NULL</code>
<code>text_quality_state</code>	<code>text</code>	No	<code>NOT_EVALUATED</code>
<code>quality_signals</code>	<code>jsonb</code>	No	<code>{}</code>
<code>ocr_decision</code>	<code>text</code>	No	<code>NOT_EVALUATED</code>

Column	Type	Null	Default
<code>ocr_decision_reason</code>	<code>text</code>	Yes	<code>NULL</code>
<code>ocr_quality_metadata</code>	<code>jsonb</code>	No	<code>{}</code>
<code>created_at</code>	<code>timestampz</code>	No	<code>now()</code>
<code>updated_at</code>	<code>timestampz</code>	No	<code>now()</code>

Constraints:

- Unique `(document_id, page_number)`
- `page_number BETWEEN 1 AND document.page_count`, enforced transactionally
- Rotation in 0, 90, 180, 270
- Primary artifact must be direct or OCR artifact from the same page/attempt

JSONB reasons:

- Quality signals and OCR metadata vary by selected parser/OCR approach.
- They are diagnostic, not business truth.
- Schema validation is versioned by producing configuration.
- No index required.

16. Pipeline-configuration and version tables

Recommendation

Use one immutable `pipeline_versions` record containing explicit component identifiers and a complete configuration snapshot.

Separate normalized configuration tables would create a configuration-management product without operational value.

16.1 `pipeline_versions`

Column	Type	Null	Default
<code>id</code>	<code>uuid</code>	No	<code>generated</code>
<code>version_label</code>	<code>text</code>	No	<code>—</code>
<code>status</code>	<code>text</code>	No	<code>DRAFT</code>
<code>parser_identity</code>	<code>text</code>	No	<code>—</code>
<code>ocr_identity</code>	<code>text</code>	No	<code>—</code>

Column	Type	Null	Default
ai_provider_identity	text	No	—
ai_model_identity	text	No	—
instruction_version	text	No	—
canonical_contract_version	text	No	—
context_preparation_version	text	No	—
normalizer_version	text	No	—
validator_version	text	No	—
evidence_generator_version	text	No	—
warning_rules_version	text	No	—
configuration_snapshot	jsonb	No	—
candidate_schema_hash	text	No	—
created_by	uuid	No	—
created_at	timestampz	No	now()
approved_at	timestampz	Yes	NULL
retired_at	timestampz	Yes	NULL

Statuses:

- DRAFT
- EVALUATING
- APPROVED
- RETIRED

Constraints:

- Unique version_label
- Approved requires approved_at
- No secrets in snapshot
- Approved records immutable except retirement status

17. Candidate tables

17.1 Storage options

One JSONB candidate

Simple and contract-friendly, but weak for:

- policy search
- review item relationships
- evidence links
- vehicle associations
- field-level corrections

Fully relational

Strong queries but can duplicate provenance columns repeatedly.

Hybrid

Relational candidate records plus one immutable snapshot artifact.

Recommendation

Use the hybrid approach.

Relational candidate tables are authoritative for review. An immutable JSONB snapshot provides reproducibility and Stored Result compatibility.

18. Candidate policy-level fields

18.1 `extraction_candidates`

Column	Type	Null	Default
<code>id</code>	<code>uuid</code>	No	generated
<code>attempt_id</code>	<code>uuid</code>	No	—
<code>document_id</code>	<code>uuid</code>	No	—
<code>contract_version</code>	<code>text</code>	No	—
<code>document_classification</code>	<code>text</code>	No	—
<code>policy_count_assessment</code>	<code>text</code>	No	—

Column	Type	Null	Default
snapshot_artifact_id	uuid	No	—
published_at	timestampz	No	now()
critical_warning_count	integer	No	0
important_warning_count	integer	No	0
information_warning_count	integer	No	0

Constraints:

- Unique attempt_id
- Candidate immutable after insertion
- Document must match attempt document

18.2 candidate_policy_fields

One row per candidate.

Column	Type	Null
candidate_id	uuid PK/FK	No
carrier_source	text	Yes
carrier_normalized	text	Yes
carrier_state	text	No
policy_number_source	text	Yes
policy_number_normalized	text	Yes
policy_number_state	text	No
effective_date_source	text	Yes
effective_date	date	Yes
effective_date_state	text	No
expiration_date_source	text	Yes
expiration_date	date	Yes
expiration_date_state	text	No
total_premium_source	text	Yes
total_premium_amount	numeric(14,2)	Yes

Column	Type	Null
premium_currency	text	Yes
total_premium_state	text	No
primary_named_insured_id	uuid	Yes
mailing_address_source	text	Yes
mailing_address_line1	text	Yes
mailing_address_line2	text	Yes
mailing_address_city	text	Yes
mailing_address_state_code	text	Yes
mailing_address_postal_code	text	Yes
mailing_address_country_code	text	Yes
mailing_address_state	text	No

Field state:

- PRESENT
- MISSING
- UNCLEAR
- CONFLICTING

Constraints:

- Nonnegative premium
- Primary insured must belong to same candidate, enforced after insured insertion
- Currency required when premium exists

Warnings and validation remain in separate tables.

19. Candidate named-insured tables

Recommendation

Primary and additional named insureds belong in the same table with an explicit role.

This avoids separate schemas and supports ordering.

19.1 candidate_named_insureds

Column	Type	Null	Rules
id	uuid	No	PK
candidate_id	uuid	No	FK
insured_role	text	No	PRIMARY, ADDITIONAL
item_order	smallint	No	Positive
source_representation	text	Yes	—
normalized_full_name	text	Yes	—
presence_state	text	No	Controlled
duplicate_of_insured_id	uuid	Yes	Self FK
created_at	timestampz	No	now()

Constraints:

- Unique (candidate_id, item_order)
- Partial unique one PRIMARY per candidate
- No automatic relationship to drivers
- Immutable

Customer conflict is represented by a validation signal, not a column.

20. Candidate vehicle tables

20.1 candidate_vehicles

Column	Type	Null
id	uuid	No
candidate_id	uuid	No
item_order	smallint	No
year_source	text	Yes
vehicle_year	smallint	Yes
year_state	text	No
make_source	text	Yes

Column	Type	Null
make_normalized	text	Yes
make_state	text	No
model_source	text	Yes
model_normalized	text	Yes
model_state	text	No
vin_source	text	Yes
vin_normalized	text	Yes
vin_state	text	No
vin_visibility	text	No
comprehensive_source	text	Yes
comprehensive_amount	numeric(14,2)	Yes
comprehensive_state	text	No
collision_source	text	Yes
collision_amount	numeric(14,2)	Yes
collision_state	text	No
currency	text	Yes
duplicate_of_vehicle_id	uuid	Yes
association_state	text	No
created_at	timestampz	No

VIN visibility:

- FULL
- PARTIAL
- MASKED
- MISSING

Association state:

- CLEAR
- AMBIGUOUS
- CONFLICTING

Constraints:

- Unique (candidate_id, item_order)
 - Year within a broad plausible database range; tighter logic in validator
 - Deductible nonnegative
 - Duplicate target must belong to same candidate
 - Deductibles are stored on the vehicle row, preventing attachment to another vehicle without changing that row
-

21. Candidate driver tables

21.1 candidate_drivers

Column	Type	Null
id	uuid	No
candidate_id	uuid	No
item_order	smallint	No
source_representation	text	Yes
normalized_full_name	text	Yes
presence_state	text	No
duplicate_of_driver_id	uuid	Yes
created_at	timestampz	No

Constraints:

- Unique (candidate_id, item_order)
 - Driver names are not derived from named insureds
 - Immutable
-

22. Candidate coverage tables

22.1 Representation

Use one table for policy-level coverage rows.

Vehicle deductibles remain on candidate_vehicles because that is the clearest ownership boundary.

22.2 candidate_coverages

Column	Type	Null
id	uuid	No
candidate_id	uuid	No
coverage_type	text	No
source_representation	text	Yes
coverage_abbreviation	text	Yes
limit_style	text	No
per_person_amount	numeric(14,2)	Yes
per_accident_amount	numeric(14,2)	Yes
combined_limit_amount	numeric(14,2)	Yes
single_amount	numeric(14,2)	Yes
currency	text	Yes
presence_state	text	No
association_state	text	No
created_at	timestampz	No

Coverage types:

- BODILY_INJURY_LIABILITY
- PROPERTY_DAMAGE_LIABILITY
- UNINSURED_UNDERINSURED_MOTORIST

Limit styles:

- SPLIT
- COMBINED_SINGLE_LIMIT
- SINGLE_AMOUNT
- MISSING
- UNCLEAR

Constraints:

- At most one active row per candidate and coverage type unless state is CONFLICTING
- Amounts nonnegative
- Split requires per-person and per-accident values
- Combined style requires combined amount

- Coverage remains policy-level

23. Candidate alternatives and conflicts

Recommendation

Use one generic alternatives table.

Field-specific child tables would create many nearly identical structures. Warning metadata alone is insufficient because the review UX must show candidate choices and evidence.

23.1 candidate_alternatives

Column	Type	Null
id	uuid	No
candidate_id	uuid	No
target_kind	text	No
target_id	uuid	Yes
field_key	text	No
alternative_order	smallint	No
source_value	text	No
normalized_value	jsonb	Yes
reason_category	text	No
created_at	timestampz	No

Target kinds:

- POLICY
- NAMED_INSURED
- VEHICLE
- DRIVER
- COVERAGE

JSONB justification:

- Alternative normalized values can be text, date, money, address, or structured limit.
- The field contract validates shape according to `field_key`.
- It is immutable and not directly approved.
- No JSON index is required.

Constraints:

- Unique target/order
- Backend verifies target belongs to candidate
- `target_id` is null only for policy-level target

24. Evidence tables

24.1 `candidate_evidence`

Column	Type	Null
<code>id</code>	<code>uuid</code>	No
<code>candidate_id</code>	<code>uuid</code>	No
<code>target_kind</code>	<code>text</code>	No
<code>target_id</code>	<code>uuid</code>	Yes
<code>field_key</code>	<code>text</code>	No
<code>document_page_id</code>	<code>uuid</code>	No
<code>source_type</code>	<code>text</code>	No
<code>supporting_excerpt</code>	<code>text</code>	Yes
<code>source_representation</code>	<code>text</code>	Yes
<code>location_geometry</code>	<code>jsonb</code>	Yes
<code>has_exact_location</code>	<code>boolean</code>	No
<code>verification_status</code>	<code>text</code>	No
<code>is_primary</code>	<code>boolean</code>	No
<code>relationship_context</code>	<code>text</code>	Yes
<code>evidence_generator_version</code>	<code>text</code>	No
<code>created_at</code>	<code>timestampz</code>	No

Geometry JSONB:

```
{
  coordinate_space_version,
  page_width,
  page_height,
```

```

    rectangles: [
      {x, y, width, height}
    ]
  }

```

Reason for JSONB:

- Geometry may contain zero, one, or several rectangles.
- Different OCR providers expose different source geometry.
- It remains provider-neutral through a normalized coordinate contract.
- Shape is validated by evidence format version.
- No spatial database extension is needed.

Constraints:

- `has_exact_location = false` requires null geometry
- Page belongs to candidate's document
- Target belongs to candidate
- Exact coordinates cannot be inferred from page-only evidence
- At most one primary evidence per target and field
- Evidence immutable

24.2 candidate_alternative_evidence

Links alternatives to evidence.

Column	Type	Null
alternative_id	uuid	No
evidence_id	uuid	No

Composite primary key.

25. Validation-signal tables

25.1 validation_signals

Column	Type	Null
id	uuid	No
candidate_id	uuid	No
target_kind	text	No
target_id	uuid	Yes

Column	Type	Null
field_key	text	Yes
category	text	No
result	text	No
severity_recommendation	text	Yes
reason_code	text	No
summary	text	No
observed_values	jsonb	No
related_customer_value	jsonb	Yes
evidence_id	uuid	Yes
publication_impact	text	No
validator_version	text	No
created_at	timestampz	No

Results:

- PASSED
- FAILED
- NOT_APPLICABLE

Publication impact:

- NONE
- WARNING
- BLOCK_PUBLICATION

JSONB stores provider-neutral observed value snapshots, not authoritative candidate data.

26. Warning tables

26.1 Separate generated warning and resolution

Recommendation: Preserve generated warnings immutably and store reviewer resolution separately.

A mutable warning row would lose what the pipeline originally found.

26.2 generated_warnings

Column	Type	Null
id	uuid	No
candidate_id	uuid	No
target_kind	text	No
target_id	uuid	Yes
field_key	text	Yes
severity	text	No
category	text	No
explanation_key	text	No
explanation_summary	text	No
blocks_approval	boolean	No
allowed_resolutions	text[]	No
primary_evidence_id	uuid	Yes
warning_rules_version	text	No
created_at	timestamptz	No

Array justification:

- Resolution options are a small controlled set.
- They are not independently queried.
- They are validated against warning category.

26.3 warning_signal_links

Column	Type	Null
warning_id	uuid	No
validation_signal_id	uuid	No

Composite primary key.

26.4 warning_resolutions

Column	Type	Null
id	uuid	No
review_id	uuid	No
warning_id	uuid	No
resolution	text	No
resolution_note	text	Yes
resolved_by	uuid	No
resolved_at	timestampz	No
review_version_at_resolution	bigint	No
is_current	boolean	No
stale_at	timestampz	Yes
row_version	bigint	No
created_at	timestampz	No
updated_at	timestampz	No

Constraints:

- Partial unique current resolution per (review_id, warning_id)
- Resolution must be allowed by warning definition
- Review and warning must share candidate baseline
- Finalized reviews freeze current resolutions

27. Review tables

27.1 reviews

Column	Type	Null	Default
id	uuid	No	generated
agency_id	uuid	No	—
demo_session_id	uuid	Yes	NULL
customer_id	uuid	No	—

Column	Type	Null	Default
document_id	uuid	No	—
origin	text	No	—
baseline_candidate_id	uuid	Yes	NULL
baseline_policy_version_id	uuid	Yes	NULL
status	text	No	OPEN
opened_by	uuid	No	—
opened_at	timestampz	No	now()
last_saved_at	timestampz	Yes	NULL
current_attempt_id	uuid	Yes	NULL
customer_version_at_open	bigint	No	—
current_policy_version_at_open	uuid	Yes	NULL
readiness_state	text	No	NOT_READY
unresolved_critical_count	integer	No	0
unresolved_important_count	integer	No	0
correction_count	integer	No	0
approved_policy_version_id	uuid	Yes	NULL
rejection_reason	text	Yes	NULL
rejection_note	text	Yes	NULL
completed_by	uuid	Yes	NULL
completed_at	timestampz	Yes	NULL
stale_reason	text	Yes	NULL
abandoned_at	timestampz	Yes	NULL
row_version	bigint	No	1
created_at	timestampz	No	now()
updated_at	timestampz	No	now()

Constraints:

- Exactly one of candidate or policy baseline
- Candidate origin requires candidate baseline

- Correction origin requires approved policy baseline
- Unique active review per baseline
- Final status requires completion fields
- Approved status requires approved policy version
- Rejected status requires reason
- Demo context must match customer and document

Readiness storage

Store a cached summary for efficient lists and approval UX.

The backend recalculates on every review mutation and again during approval.

The cached state is not authoritative for approval by itself.

28. Review-correction tables

Options

Append every edit only

Strong history but difficult current-state queries.

Current correction only plus audit

Simple but loses structured edit sequence.

Hybrid

Append correction rows and mark one current.

Recommendation

Use hybrid.

28.1 `review_field_corrections`

Column	Type	Null
<code>id</code>	<code>uuid</code>	No
<code>review_id</code>	<code>uuid</code>	No
<code>target_kind</code>	<code>text</code>	No
<code>target_id</code>	<code>uuid</code>	Yes

Column	Type	Null
field_key	text	No
sequence_number	integer	No
prior_value	jsonb	Yes
proposed_value	jsonb	No
source_representation	text	Yes
correction_reason	text	Yes
saved_by	uuid	No
saved_at	timestampz	No
is_current	boolean	No
superseded_at	timestampz	Yes
idempotency_record_id	uuid	Yes
created_at	timestampz	No

JSONB justification:

- Corrected values can be date, money, text, address, or structured coverage limit.
- Field contract validates the shape.
- Values remain review proposals, not approved truth.
- No JSON index required.

Constraints:

- Unique (review_id, target_kind, target_id, field_key, sequence_number)
- Partial unique current correction per target field
- Backend validates target belongs to review baseline
- Corrections cannot be added to terminal review

Undo

Undo supersedes the current correction with a new correction equal to the previous effective value. It does not delete history.

29. Mark Not Present and acknowledgements

29.1 review_field_decisions

Column	Type	Null
id	uuid	No
review_id	uuid	No
target_kind	text	No
target_id	uuid	Yes
field_key	text	No
decision_type	text	No
reason	text	Yes
saved_by	uuid	No
saved_at	timestampz	No
review_version_at_decision	bigint	No
is_current	boolean	No
stale_at	timestampz	Yes
created_at	timestampz	No

Decision types:

- MARK_NOT_PRESENT
- CONFIRM_IDENTITY_DIFFERENCE
- CONFIRM_ADDRESS_DIFFERENCE

Constraints:

- Current decision unique per field
- Required reason for permitted Critical difference
- Empty candidate value is not equivalent to a decision
- Invalidated when baseline changes

Warning acknowledgements remain in `warning_resolutions`.

30. Reviewer-added and removed items

Recommendation

Use review-specific item-change tables.

Reusing candidate tables would mutate or confuse the immutable candidate.

30.1 `review_vehicle_changes`

Column	Type	Null
<code>id</code>	<code>uuid</code>	No
<code>review_id</code>	<code>uuid</code>	No
<code>change_type</code>	<code>text</code>	No
<code>baseline_vehicle_id</code>	<code>uuid</code>	Yes
<code>final_order</code>	<code>smallint</code>	Yes
<code>vehicle_year</code>	<code>smallint</code>	Yes
<code>make</code>	<code>text</code>	Yes
<code>model</code>	<code>text</code>	Yes
<code>vin</code>	<code>text</code>	Yes
<code>vin_visibility</code>	<code>text</code>	Yes
<code>comprehensive_amount</code>	<code>numeric(14,2)</code>	Yes
<code>collision_amount</code>	<code>numeric(14,2)</code>	Yes
<code>currency</code>	<code>text</code>	Yes
<code>evidence_page_id</code>	<code>uuid</code>	Yes
<code>reviewer_note</code>	<code>text</code>	Yes
<code>created_by</code>	<code>uuid</code>	No
<code>created_at</code>	<code>timestampz</code>	No
<code>updated_at</code>	<code>timestampz</code>	No
<code>row_version</code>	<code>bigint</code>	No

Change types:

- `ADD`

- REMOVE

Rules:

- Add has no baseline item and requires useful vehicle identity.
- Remove requires baseline item and no replacement scalar values.
- Candidate vehicle edits use field corrections.

30.2 review_driver_changes

Same structure with:

- baseline driver
- full name
- source representation
- final order
- evidence page
- note

30.3 review_named_insured_changes

Supports additional named insured additions/removals.

Primary insured changes use field correction rather than item removal.

31. Approval-readiness calculation contract

Inputs

- Review status
- Review row version
- Baseline candidate or policy version
- Current document state
- Current reviewable attempt
- Active attempt
- Required field effective values
- Field decisions
- Generated warnings
- Current warning resolutions
- Added and removed items
- Customer version
- Customer-name conflict
- Customer-address conflict
- Current-policy identity
- Optional missing values
- Stale reason

Derived effective value order

1. Current reviewer correction
2. Current Mark Not Present decision
3. Reviewer item change
4. Baseline candidate or approved value

Approval blockers

- Review stale
- Terminal review
- Missing unresolved required field
- Unresolved Critical warning
- Unresolved Important warning
- Invalid correction
- Customer identity difference not confirmed
- Current policy changed
- Active processing result superseded baseline
- Unsupported document requiring rejection
- Invalid relationship
- Approval context incomplete

Calculation

- Calculate on every successful review mutation.
- Store cached counts and `readiness_state`.
- Recalculate from source records inside approval transaction.
- Do not accept a frontend Ready value.

Invalidation

- Correction
- Decision
- Warning resolution
- Item add/remove
- Customer edit
- Current-policy replacement
- New selected processing attempt
- Baseline change
- Validation-rule change only for a new attempt, not silently on existing review

Response

```
{
  review_id,
  version,
  readiness: READY | NOT_READY | STALE,
  unresolved_critical_count,
```

```

unresolved_important_count,
unsaved_client_changes: not supplied by server,
blockers: [
  {
    code,
    target,
    message,
    action
  }
]
}

```

32. Approved-policy-version tables

Recommendation

Copy final reviewed values into dedicated immutable approved tables.

Referencing editable review records would make approved history depend on mutable data.

32.1 `policy_versions`

Column	Type	Null
<code>id</code>	<code>uuid</code>	No
<code>agency_id</code>	<code>uuid</code>	No
<code>demo_session_id</code>	<code>uuid</code>	Yes
<code>customer_id</code>	<code>uuid</code>	No
<code>source_document_id</code>	<code>uuid</code>	No
<code>source_attempt_id</code>	<code>uuid</code>	Yes
<code>source_review_id</code>	<code>uuid</code>	No
<code>pipeline_version_id</code>	<code>uuid</code>	Yes
<code>correction_of_policy_version_id</code>	<code>uuid</code>	Yes
<code>replacement_of_policy_version_id</code>	<code>uuid</code>	Yes
<code>carrier_name</code>	<code>text</code>	No
<code>policy_number</code>	<code>text</code>	No
<code>effective_date</code>	<code>date</code>	No

Column	Type	Null
expiration_date	date	No
total_premium_amount	numeric(14,2)	Yes
currency	text	Yes
mailing_address_line1	text	Yes
mailing_address_line2	text	Yes
mailing_address_city	text	Yes
mailing_address_state_code	text	Yes
mailing_address_postal_code	text	Yes
mailing_address_country_code	text	Yes
approved_by	uuid	No
approved_at	timestampz	No
approval_idempotency_record_id	uuid	No
created_at	timestampz	No

Constraints:

- Unique source_review_id
- Effective date <= expiration date
- Total premium nonnegative
- Immutable through permissions and backend rules
- Correction relationship cannot reference itself
- Source relationships must share customer and document context

32.2 policy_version_named_insureds

- id uuid
- policy_version_id uuid
- insured_role text
- item_order smallint
- full_name text
- source_representation text nullable

Partial unique one primary insured.

32.3 policy_version_vehicles

- identity and order
- year

- make
- model
- VIN
- VIN visibility
- comprehensive deductible
- collision deductible
- currency
- source representation snapshots where useful

32.4 `policy_version_drivers`

- identity
- order
- full name
- source representation

32.5 `policy_version_coverages`

- coverage type
- source representation
- limit style
- per-person
- per-accident
- combined or single amount
- currency

All approved child rows are immutable.

33. Current-policy selection

Alternatives

Status column on policy version

Simple query but mutates an otherwise immutable version.

Current-policy FK on customer

Simple but couples customer row changes to policy selection and complicates audit metadata.

Separate relationship table

Keeps policy versions immutable and makes selection explicit.

Recommendation

Use a separate table.

33.1 `customer_current_policies`

Column	Type	Null
<code>customer_id</code>	<code>uuid</code>	No
<code>policy_version_id</code>	<code>uuid</code>	No
<code>selected_by</code>	<code>uuid</code>	No
<code>selected_at</code>	<code>timestampz</code>	No
<code>previous_policy_version_id</code>	<code>uuid</code>	Yes
<code>row_version</code>	<code>bigint</code>	No
<code>updated_at</code>	<code>timestampz</code>	No

Constraints:

- Primary key `customer_id`
- Unique `policy_version_id`
- Selected policy must belong to customer, transactionally enforced
- One row guarantees one Current policy

Behaviors

- First approval inserts selection.
- Replacement updates selection atomically.
- Historical approval does not change it.
- Correction Review normally replaces selection.
- Failed replacement leaves row unchanged.
- No normal staff deletion.

34. Approval transaction

Atomic sequence

1. Resolve actor and agency.
2. Load review for update.
3. Verify expected review version.
4. Verify status is Open or Ready.
5. Verify baseline candidate or approved version.
6. Recalculate effective review values.
7. Recalculate readiness.
8. Verify customer row version.
9. Lock current-policy selection context.
10. Verify expected current policy.

11. Create immutable policy version.
12. Create named insureds.
13. Create vehicles.
14. Create drivers.
15. Create coverages.
16. Insert or update current-policy selection when requested.
17. Optionally update customer contact address and increment customer version.
18. Mark review Approved.
19. Set document disposition Approved.
20. Clear current reviewable attempt if appropriate.
21. Create same-transaction audit events.
22. Complete idempotency record.
23. Commit.

Idempotency

Same actor, route, key, and request fingerprint returns the original approval response.

A reused key with different request content returns conflict.

Concurrency

The request includes:

- expected review version
- expected customer version
- expected current policy ID or explicit null
- expected current-policy selection version

Mismatch returns `409 CURRENT_POLICY_CHANGED` or `409 REVIEW_STALE`.

Rollback

Any failure leaves:

- no new policy version
- prior Current unchanged
- customer contact unchanged
- review unapproved
- no false approval event

Repeated approval

If review is already approved and the idempotency identity matches, return the existing policy version.

Otherwise return `ALREADY_APPROVED`.

35. Rejection records and transaction

Rejection reasons

- `WRONG_CUSTOMER`
- `UNSUPPORTED_DOCUMENT`
- `MULTIPLE_POLICIES`
- `UNREADABLE_DOCUMENT`
- `INCORRECT_OR_INCOMPLETE_RESULT`
- `REPLACEMENT_REQUIRED`
- `OTHER`

Transaction

1. Verify actor.
2. Lock review.
3. Verify expected version.
4. Ensure not approved or rejected.
5. Set Rejected fields.
6. Preserve customer and Current policy.
7. Update document disposition Rejected when no approved outcome exists.
8. Create audit event.
9. Complete idempotency record.

After rejection:

- Candidate remains immutable.
- Review records become read-only.
- A new Technical Retry, Reprocessing, or replacement upload is separate.
- Rejection cannot be directly reversed.

36. Correction Review persistence

Correction Review reuses:

- `reviews`
- `review_field_corrections`
- `review_field_decisions`
- review item-change tables
- `warning_resolutions` when correction-specific warnings exist
- approval transaction
- approved policy tables

Differences:

- `origin = CORRECTION`

- `baseline_policy_version_id` is populated
- no baseline candidate
- no processing attempt
- no Document AI usage reservation
- effective values begin from approved version
- new version records `correction_of_policy_version_id`
- Current policy remains unchanged until approval

Abandonment:

- Mark review Abandoned.
- Preserve review history.
- Do not alter original policy.

37. Audit-event tables

37.1 `audit_events`

Column	Type	Null
<code>id</code>	<code>uuid</code>	No
<code>agency_id</code>	<code>uuid</code>	No
<code>demo_session_id</code>	<code>uuid</code>	Yes
<code>actor_type</code>	<code>text</code>	No
<code>actor_user_id</code>	<code>uuid</code>	Yes
<code>event_type</code>	<code>text</code>	No
<code>occurred_at</code>	<code>timestampz</code>	No
<code>customer_id</code>	<code>uuid</code>	Yes
<code>document_id</code>	<code>uuid</code>	Yes
<code>attempt_id</code>	<code>uuid</code>	Yes
<code>review_id</code>	<code>uuid</code>	Yes
<code>policy_version_id</code>	<code>uuid</code>	Yes
<code>email_delivery_id</code>	<code>uuid</code>	Yes
<code>summary</code>	<code>text</code>	No
<code>details</code>	<code>jsonb</code>	No
<code>correlation_id</code>	<code>uuid</code>	No

Column	Type	Null
event_version	integer	No
created_at	timestampz	No

Actor types:

- STAFF
- DEMO_USER
- SYSTEM
- WORKER

JSONB justification:

- Event details vary by business event.
- It holds small structured before/after snapshots and reason codes.
- Event type and event version define validation.
- It is not used as business state.
- GIN indexing is unnecessary initially.

Same-transaction events:

- customer creation/edit
- correction
- warning acknowledgement
- approval
- rejection
- Current-policy replacement
- email-send request
- retry/reprocessing request

Background events:

- processing completed
- processing failed
- email sent/failed
- demo expired

PII minimization:

- Store only values necessary to explain material changes.
- Avoid complete source text.
- Mask policy identifiers in summaries.
- Full old/new values may be restricted in detailed audit access.

Demo cleanup may hard-delete demo audit events with the session because they are disposable synthetic data.

38. Usage and cost tables

38.1 `usage_limits`

Column	Type	Null
<code>id</code>	<code>uuid</code>	No
<code>agency_id</code>	<code>uuid</code>	No
<code>demo_session_id</code>	<code>uuid</code>	Yes
<code>environment_kind</code>	<code>text</code>	No
<code>limit_category</code>	<code>text</code>	No
<code>unit</code>	<code>text</code>	No
<code>period_start</code>	<code>timestampz</code>	No
<code>period_end</code>	<code>timestampz</code>	No
<code>maximum_quantity</code>	<code>numeric(18,6)</code>	Yes
<code>maximum_estimated_cost</code>	<code>numeric(14,6)</code>	Yes
<code>currency</code>	<code>text</code>	Yes
<code>enabled</code>	<code>boolean</code>	No
<code>created_at</code>	<code>timestampz</code>	No
<code>updated_at</code>	<code>timestampz</code>	No
<code>row_version</code>	<code>bigint</code>	No

No fixed cost values are invented.

38.2 `usage_reservations`

Column	Type	Null
<code>id</code>	<code>uuid</code>	No
<code>agency_id</code>	<code>uuid</code>	No
<code>demo_session_id</code>	<code>uuid</code>	Yes
<code>attempt_id</code>	<code>uuid</code>	No
<code>limit_id</code>	<code>uuid</code>	No
<code>reserved_quantity</code>	<code>numeric(18,6)</code>	No

Column	Type	Null
reserved_cost	numeric(14,6)	Yes
currency	text	Yes
status	text	No
expires_at	timestamptz	No
finalized_at	timestamptz	Yes
created_at	timestamptz	No
updated_at	timestamptz	No

Statuses:

- RESERVED
- FINALIZED
- RELEASED
- EXPIRED

38.3 usage_records

Column	Type	Null
id	uuid	No
agency_id	uuid	No
demo_session_id	uuid	Yes
attempt_id	uuid	Yes
email_delivery_id	uuid	Yes
category	text	No
mode	text	No
provider_capability	text	Yes
provider_identity	text	Yes
model_or_processor_identity	text	Yes
quantity	numeric(18,6)	No
unit	text	No
estimated_cost	numeric(14,6)	Yes
currency	text	Yes

Column	Type	Null
cost_known	boolean	No
provider_usage_metadata	jsonb	No
occurred_at	timestampz	No

Constraints:

- Nonnegative quantity and cost
- Stored Result uses `STORED`
- Simulated email uses `SIMULATED`
- No provider-specific business columns

Reservation transaction

Attempt and job creation occur only after allowance is locked and reserved.

Concurrent requests cannot reserve beyond the same configured limit.

39. Email-delivery tables

39.1 email_deliveries

Column	Type	Null
id	uuid	No
agency_id	uuid	No
demo_session_id	uuid	Yes
policy_version_id	uuid	No
customer_id	uuid	No
recipient_snapshot	text	No
content_version	text	No
status	text	No
requested_by	uuid	No
requested_at	timestampz	No
sent_at	timestampz	Yes
failed_at	timestampz	Yes

Column	Type	Null
provider_reference	text	Yes
product_idempotency_record_id	uuid	No
provider_idempotency_identity	text	Yes
resend_of_email_delivery_id	uuid	Yes
failure_category	text	Yes
diagnostic_reference	text	No
row_version	bigint	No
created_at	timestampz	No
updated_at	timestampz	No

Constraints:

- Policy version must belong to customer
- Demo sends use `SIMULATED`
- Recipient is backend-derived
- Approval is never changed by email status

40. Demo-session tables

Alternatives

Session FK on every record

Safe but repetitive.

One tenant/workspace boundary

Becomes generalized multi-tenancy.

Schema per session

Operationally expensive.

Database per session

Unnecessary and costly.

Recommendation

Use `demo_session_id` on top-level disposable aggregates:

- customers
- documents
- attempts through document
- reviews
- policy versions
- audit events
- usage
- email

Child rows inherit isolation through their parent.

40.1 `demo_sessions`

Column	Type	Null
<code>id</code>	<code>uuid</code>	No
<code>agency_id</code>	<code>uuid</code>	No
<code>demo_actor_user_id</code>	<code>uuid</code>	No
<code>template_set_id</code>	<code>uuid</code>	No
<code>status</code>	<code>text</code>	No
<code>reset_generation</code>	<code>integer</code>	No
<code>created_at</code>	<code>timestampz</code>	No
<code>last_activity_at</code>	<code>timestampz</code>	No
<code>expires_at</code>	<code>timestampz</code>	No
<code>reset_started_at</code>	<code>timestampz</code>	Yes
<code>cleanup_started_at</code>	<code>timestampz</code>	Yes
<code>cleaned_at</code>	<code>timestampz</code>	Yes
<code>failure_reference</code>	<code>text</code>	Yes
<code>row_version</code>	<code>bigint</code>	No

Session filters are mandatory in backend data access for demo users.

A demo record may never reference a staff customer or staff document.

41. Synthetic template and Stored Result tables

41.1 demo_template_sets

Column	Type	Null
id	uuid	No
version_label	text	No
status	text	No
synthetic_seed_snapshot	jsonb	No
contract_version	text	No
created_at	timestampz	No
published_at	timestampz	Yes
retired_at	timestampz	Yes

JSONB is justified because:

- It is an immutable seed template, not operational business truth.
- It is instantiated into relational customer, document, review, and policy rows.
- A versioned seed validator checks compatibility.
- No JSON index is required.

41.2 stored_result_templates

Column	Type	Null
id	uuid	No
template_set_id	uuid	No
version_label	text	No
display_name	text	No
source_file_id	uuid	No
pipeline_version_id	uuid	No
candidate_contract_version	text	No
candidate_snapshot	jsonb	No
evidence_snapshot	jsonb	No
warnings_snapshot	jsonb	No

Column	Type	Null
<code>stage_plan</code>	<code>jsonb</code>	No
<code>policy_history_snapshot</code>	<code>jsonb</code>	No
<code>simulated_usage_snapshot</code>	<code>jsonb</code>	No
<code>status</code>	<code>text</code>	No
<code>created_at</code>	<code>timestampz</code>	No
<code>published_at</code>	<code>timestampz</code>	Yes
<code>retired_at</code>	<code>timestampz</code>	Yes

Templates are immutable after publication.

Replay behavior

- Create a real document or sample workflow instance.
- Create a real Stored Result attempt.
- Instantiate relational candidate, evidence, warnings, and review.
- Create real reviewer edits and approval later.
- Use simulated usage.
- Simulate email.
- Never call live OCR or AI.

42. Benchmark and evaluation records

Recommendation

Do not build benchmark tables into the production product database.

Keep:

- benchmark manifests
- ground truth
- evaluation results
- comparison reports
- repeatability reports

as version-controlled structured files plus private benchmark artifacts.

Store only:

- approved `pipeline_versions`
- selected Stored Result templates

- optional release decision reference in pipeline configuration snapshot

Why

The evaluation process needs reproducibility, not a product dashboard.

A future dedicated evaluation store is justified only when evaluation volume and team size grow.

43. Retention and cleanup fields

Retention is represented through:

- retention_category
- expires_at
- cleanup_status
- cleanup_attempts
- deleted_at
- diagnostic failure reference

Required categories

Data	Retention
Original business PDF	Business
Candidate	Business
Approved policy	Business
Audit	Business
Raw AI response	Diagnostic, expiring
Raw OCR response	Diagnostic, expiring
Temporary page image	Temporary
Stored Result source	Demo template
Benchmark artifact	Evaluation
Incomplete upload	Temporary
Demo session	Expiring disposable
Old job execution detail	Diagnostic

A legal-hold feature is not required in Release 1.

A future owner-directed deletion request requires a controlled internal operation, not a staff UX action.

44. Index strategy

Index	Query supported	Type	Required
<code>customers(agency_id, archived_at, updated_at DESC, id)</code>	Customer list	B-tree	Yes
Trigram on <code>customers.search_text</code>	Partial customer search	GIN <code>pg_trgm</code>	Yes
<code>candidate_policy_fields(policy_number_normalized)</code>	Policy-number search	B-tree	Yes
<code>candidate_policy_fields(carrier_normalized)</code>	Carrier filtering	B-tree	Optional initially
<code>policy_versions(customer_id, approved_at DESC)</code>	Policy history	B-tree	Yes
<code>policy_versions(policy_number)</code>	Approved policy search	B-tree	Yes
<code>documents(agency_id, disposition, last_activity_at DESC, id)</code>	Documents and Overview	B-tree	Yes
<code>documents(customer_id, uploaded_at DESC)</code>	Customer documents	B-tree	Yes
<code>documents(source_file_id)</code> unique	One source PDF	Unique	Yes
<code>file_objects(content_sha256, byte_size)</code>	Duplicate inspection	B-tree	Yes
Partial <code>processing_attempts(document_id)</code> active statuses	One active attempt	Unique partial	Yes
<code>processing_attempts(document_id, created_at DESC)</code>	Attempt history	B-tree	Yes
<code>processing_attempts(status, last_progress_at)</code>	Stale detection	B-tree	Yes
Partial <code>background_jobs(status, available_at, priority, created_at)</code> for available jobs	Job claims	B-tree	Yes

Index	Query supported	Type	Required
<code>background_jobs(lease_expires_at)</code> where claimed/ running	Expired leases	Partial B- tree	Yes
<code>processing_stage_executions(attempt_id, stage, execution_number)</code>	Progress	Unique	Yes
<code>reviews(status, unresolved_critical_count DESC, opened_at, id)</code>	Review Queue	B-tree	Yes
<code>reviews(document_id, created_at DESC)</code>	Document review history	B-tree	Yes
<code>generated_warnings(candidate_id, severity)</code>	Review summary	B-tree	Yes
<code>candidate_evidence(candidate_id, target_kind, target_id, field_key)</code>	Review evidence	B-tree	Yes
<code>audit_events(customer_id, occurred_at DESC, id)</code>	Customer activity	B-tree	Yes
<code>audit_events(document_id, occurred_at DESC, id)</code>	Document activity	B-tree	Yes
<code>audit_events(policy_version_id, occurred_at DESC, id)</code>	Policy activity	B-tree	Yes
<code>usage_records(agency_id, occurred_at)</code>	Usage period	B-tree	Yes
<code>usage_reservations(status, expires_at)</code>	Reservation cleanup	B-tree	Yes
<code>demo_sessions(status, expires_at)</code>	Demo cleanup	B-tree	Yes
<code>email_deliveries(policy_version_id, requested_at DESC)</code>	Email history	B-tree	Yes
<code>email_deliveries(status, updated_at)</code>	Email worker/ status	B-tree	Yes
<code>idempotency_records(actor_scope_id, route_key, idempotency_key)</code>	Safe repeats	Unique	Yes

Avoid indexing raw JSONB and every foreign key indiscriminately. Add indexes when a documented query uses them.

45. Constraint and invariant inventory

Invariant	Enforcement
One active attempt per document	Partial unique database index
One Current policy per customer	<code>customer_current_policies</code> primary key
One finalized outcome per review	Transaction plus status check
Approved policy immutable	Backend permissions and no update API
Candidate immutable	Backend/worker rule and no update path
One source PDF per document	Unique FK
Technical Retry maximum two	Ordinal check, unique index, backend count
Nonnegative usage	Database checks
Evidence page belongs to document	Transaction/backend validation
Evidence target belongs to candidate	Backend validation and contract tests
Correction target belongs to review baseline	Backend validation
Demo aggregate cannot reference staff aggregate	Transaction/backend validation
Stored Result is not live	Database checks on attempt and usage mode
Email requires approved version	FK and backend transaction
Approval cannot have blocking warnings	Approval recalculation transaction
Completed attempt has no active job	Worker/backend transaction
Provider secrets absent from configuration	Deployment and review policy
Review has one baseline	Database check
One primary named insured	Partial unique index
Current policy belongs to customer	Approval transaction
Job claim has one valid owner	Database claim transaction
Repeated approval creates one policy	Idempotency plus unique source review
Source page count is 1–10	Database check
Demo template is immutable after publication	Backend permissions
Raw artifacts expire when configured	Cleanup jobs and monitoring

46. Deletion and archival behavior

Record	Staff delete?	Internal behavior
Customer	No	Archive or controlled future deletion
Document	No	Preserve; future controlled deletion may remove source and dependent data
Original PDF	No	Delete only through retention/customer-data process
Failed upload	No UX	Clean orphan file after expiry
Failed attempt	No	Preserve
Raw provider artifact	No	Hard-delete after expiry
Approved policy version	No	Preserve
Audit event	No	Preserve, except disposable demo session
Demo session	Reset/expire	Hard-delete disposable records
Benchmark artifact	Owner-controlled	Controlled deletion
Email record	No	Preserve delivery history
Idempotency record	No	Expire after retention window
Job execution	No	Prune after diagnostic retention

Soft versus hard deletion

- Business aggregates use archival or controlled deletion.
- Temporary, diagnostic, and demo data may be hard-deleted.
- Do not add `deleted_at` to every child table.
- Referential deletion order must be explicit.
- Storage deletion and relational deletion are coordinated through cleanup jobs.

47. API style and conventions

47.1 Base path

`/api/v1`

47.2 Media type

`application/json`

Signed Storage transfer uses provider-required binary transfer outside business endpoints.

47.3 Authentication

Authorization: Bearer <Supabase access token>

47.4 Public identifiers

UUID strings.

47.5 Time and date

- Dates: YYYY-MM-DD
- Timestamps: ISO 8601 UTC with Z

47.6 Field naming

Lowercase snake_case JSON fields.

47.7 Monetary values

```
{  
  amount: "1250.00",  
  currency: "USD"  
}
```

Use decimal strings to avoid floating-point ambiguity.

47.8 Missing and null

- Missing request property: caller did not supply a change.
- Explicit null: clear an optional value when permitted.
- Response null: value absent.
- Candidate state explains whether absent, unclear, or conflicting.

47.9 Enum values

Uppercase stable strings.

47.10 Request and correlation identifiers

Response headers:

- X-Request-ID
- X-Correlation-ID when part of a workflow

47.11 Idempotency

`Idempotency-Key` header for required routes.

47.12 Concurrency

Use `If-Match` with an opaque version token:

```
If-Match: "rv:17"
```

Responses provide:

```
ETag: "rv:18"
```

47.13 Pagination

Cursor response:

```
{
  items: [],
  page: {
    next_cursor: "opaque-or-null",
    has_more: true
  }
}
```

Maximum page size: 100.

Default operational list page size: 25.

47.14 Sorting

Query:

```
sort=-last_activity_at
```

Only documented fields are accepted.

Stable secondary sort by UUID.

47.15 Error envelope

Defined in Section 65.

48. Authentication and authorization API contract

Frontend identity

The frontend sends the Supabase-issued access token.

Backend verification

FastAPI verifies:

- signature
- issuer
- audience
- expiry
- required subject

Then resolves:

- `app_users`
- active membership
- agency
- active demo session when applicable

Outcomes

- Missing/expired token: `401 UNAUTHENTICATED`
- Valid identity without active membership: `403 FORBIDDEN`
- Object outside context: `404 NOT_FOUND` to avoid unnecessary disclosure
- Demo-prohibited action: `403 DEMO_RESTRICTED`

Signed files

A valid user token alone is insufficient. The backend checks the requested file's relationship to:

- agency
- demo session
- customer
- document
- policy or template access

No password-reset or invitation APIs are added.

49. Customer API

49.1 List customers

GET /api/v1/customers

Query:

- q
- attention
- has_current_policy
- sort
- cursor
- limit

Response row:

- customer ID
- full name
- email
- phone
- current-policy summary
- expiration
- attention summary
- last activity
- version token

Status: 200

49.2 Duplicate check

POST /api/v1/customers/duplicate-check

Request:

- full name
- email
- phone

Response:

- possible matches
- match reasons
- existing policy summary

No audit event.

49.3 Create customer

POST /api/v1/customers

Requires Idempotency-Key .

Request:

- full name
- optional email
- optional phone
- structured optional address
- `duplicate_override` boolean
- acknowledged duplicate IDs when overriding

Response:

- customer resource
- warnings
- next actions

Success: 201

Errors:

- validation
- duplicate warning
- demo restriction
- idempotency conflict

Audit:

- customer created
- duplicate override when used

49.4 Get customer

GET /api/v1/customers/{customer_id}

Returns:

- contact information
- current policy summary
- attention
- document counts
- version token

49.5 Update customer

PATCH /api/v1/customers/{customer_id}

Requires If-Match .

Request contains only changed contact fields.

Success: 200

Conflict: 409 CUSTOMER_STALE

Audit material changes.

49.6 Current policy

GET /api/v1/customers/{customer_id}/current-policy

Returns 200 or 204 when none.

49.7 Policy history

GET /api/v1/customers/{customer_id}/policy-versions

Cursor pagination, newest approval first.

49.8 Customer documents

GET /api/v1/customers/{customer_id}/documents

Supports document filters.

49.9 Customer activity

GET /api/v1/customers/{customer_id}/activity

Cursor pagination and category filters.

50. Document API

50.1 List documents

GET /api/v1/documents

Filters:

- q
- status
- warning_severity
- processing_method
- origin
- duplicate
- date_from
- date_to
- customer_id
- sort
- cursor
- limit

50.2 Get document

```
GET /api/v1/documents/{document_id}
```

Returns:

- source metadata
- customer summary
- stored disposition
- derived display state
- active attempt
- current reviewable attempt
- warning summary
- approved-policy relationship
- available actions
- version token

50.3 Status

```
GET /api/v1/documents/{document_id}/status
```

Lightweight polling resource.

50.4 Processing detail

```
GET /api/v1/documents/{document_id}/processing
```

Returns active/latest attempt and stage timeline.

50.5 Attempts

```
GET /api/v1/documents/{document_id}/attempts
```

Newest first.

50.6 Source access

```
POST /api/v1/documents/{document_id}/source-access
```

Request:

- access purpose: `VIEW` or `DOWNLOAD`

Response:

- short-lived URL
- expiry
- content type
- filename

50.7 Failure detail

```
GET /api/v1/documents/{document_id}/failure
```

Returns latest relevant failed attempt.

50.8 Duplicate inspection

```
POST /api/v1/documents/duplicate-check
```

Internal upload-flow resource.

Request:

- file authorization or content hash
- customer

Response:

- same-customer matches
- cross-customer matches
- available override action

50.9 Activity

```
GET /api/v1/documents/{document_id}/activity
```

51. Signed upload API

51.1 Request authorization

POST /api/v1/uploads/authorizations

Requires Idempotency-Key.

Request:

- customer ID
- filename
- reported content type
- reported size
- optional replacement document ID
- sample-template ID only in demo mode

Backend:

- authorizes customer
- checks demo restrictions
- checks initial limits
- creates generated object path
- creates authorization record
- returns signed transfer permission

Success: 201

51.2 Transfer

Frontend uploads directly to the exact private object path.

The browser cannot supply another path.

51.3 Confirm completion

POST /api/v1/uploads/{authorization_id}/complete

Requires same or a new completion-scoped Idempotency-Key.

Request:

- storage completion token or object identity
- client-observed byte size

Backend:

1. verifies authorization
2. verifies object path
3. reads object metadata
4. validates real PDF
5. calculates content hash
6. inspects page count
7. checks duplicates
8. creates file object
9. creates document
10. creates attempt
11. reserves usage
12. creates job
13. creates audit event
14. marks authorization completed

All creation from step 8 onward is transactional.

Response: `201` with Processing document.

Duplicate handling

If duplicate needs user confirmation, completion returns:

`409 DUPLICATE_DOCUMENT_WARNING`

The authorization remains available for a limited period.

A follow-up completion may include:

- `duplicate_override = true`
- acknowledged matching document ID
- paid-processing confirmation

Cleanup

Expired or rejected objects become cleanup jobs.

52. Signed download and PDF-view API

Route

`POST /api/v1/documents/{document_id}/source-access`

Authorization

Rechecked every request.

Response

- short-lived signed URL
- expiry
- filename
- byte size
- content type
- page count
- demo-download restriction

Missing object

`503 STORAGE_UNAVAILABLE` when relational record exists but object cannot be reached.

Expired URL

Frontend requests a new URL. Signed URLs are not refreshed directly by Storage without backend authorization.

Audit

Routine viewer access is not a business audit event.

Explicit file download may create an audit event only if future customer policy requires it. It is not required in Release 1.

53. Processing-status and polling API

Recommendation

Use both:

- individual document status for Processing Detail
- combined status feed for Overview, Documents, and notifications

53.1 Individual status

`GET /api/v1/documents/{document_id}/status`

Returns:

- document ID

- stored disposition
- derived display state
- active attempt ID
- current stage
- stage description
- latest progress time
- delayed boolean
- current reviewable attempt ID
- review ID if available
- failure summary
- Stored Result boolean
- version token
- poll-after recommendation in seconds
- event identity for client deduplication

Supports `If-None-Match`.

Response may be `304 Not Modified`.

53.2 Combined feed

`GET /api/v1/processing/status-feed`

Query:

- `after`
- optional document IDs
- `limit`

Returns changed active or recently completed document states with stable event IDs.

Polling guidance

- Processing Detail: individual endpoint
- Overview and Documents: combined feed plus normal list refresh
- Review Queue: refresh on completion event and page focus
- Stop polling final states

The server's `poll_after_seconds` is guidance, not authority.

54. Review Queue API

`GET /api/v1/review-queue`

Query:

- q
- severity
- ocr_used
- has_current_policy
- minimum_age_minutes
- sort
- cursor
- limit

Default order:

1. highest unresolved severity
2. oldest ready time
3. review ID

Row:

- review ID
- document and customer
- carrier and policy number when available
- critical/important counts
- primary warning
- waiting duration
- OCR used
- current-policy indicator
- recently opened actor summary
- version token
- review URL data

Failed documents are excluded by query invariant.

55. Review API

55.1 Workspace

```
GET /api/v1/reviews/{review_id}
```

Returns:

- review metadata
- baseline information
- grouped candidate or approved values
- evidence
- warnings

- current corrections
- item changes
- readiness
- source-access action
- version token

PDF and review data remain independently loadable.

55.2 Save scalar correction

```
POST /api/v1/reviews/{review_id}/corrections
```

Requires:

- If-Match
- Idempotency-Key

Request:

- target kind
- target ID when applicable
- field key
- proposed value
- optional source representation
- optional reason

Response:

- saved correction
- updated effective field
- updated readiness
- new review version

55.3 Undo correction

```
POST /api/v1/reviews/{review_id}/corrections/{correction_id}/undo
```

Creates a new correction state.

55.4 Mark Not Present

```
POST /api/v1/reviews/{review_id}/field-decisions
```

Request:

- target
- field
- MARK_NOT_PRESENT

- required reason where applicable

55.5 Resolve warning

PUT /api/v1/reviews/{review_id}/warnings/{warning_id}/resolution

Request:

- resolution
- optional/required note

55.6 Confirm identity difference

Uses the field-decisions route with `CONFIRM_IDENTITY_DIFFERENCE`.

55.7 Add vehicle

POST /api/v1/reviews/{review_id}/vehicles

55.8 Edit added vehicle

PATCH /api/v1/reviews/{review_id}/vehicles/{change_id}

55.9 Remove candidate vehicle

POST /api/v1/reviews/{review_id}/vehicles/{candidate_vehicle_id}/remove

55.10 Undo vehicle removal

DELETE /api/v1/reviews/{review_id}/vehicle-removals/{change_id}

55.11 Driver routes

Equivalent:

- POST /reviews/{id}/drivers
- PATCH /reviews/{id}/drivers/{change_id}
- POST /reviews/{id}/drivers/{candidate_driver_id}/remove
- DELETE /reviews/{id}/driver-removals/{change_id}

55.12 Additional insured routes

Equivalent under `/named-insureds`.

55.13 Readiness

GET /api/v1/reviews/{review_id}/readiness

55.14 Refresh stale review

```
POST /api/v1/reviews/{review_id}/refresh
```

This does not merge stale edits automatically. It returns current baseline and conflict information.

55.15 Abandon Correction Review

```
POST /api/v1/reviews/{review_id}/abandon
```

Only Correction Review may be abandoned.

Write behavior

Every write:

- requires current review version
- validates target
- recalculates readiness
- increments version
- returns updated readiness
- creates material audit events
- returns `409 REVIEW_STALE` on conflict

One generic full-review JSON update is rejected because it would create large conflict surfaces and unclear audit history.

56. Approval API

Approval summary

```
GET /api/v1/reviews/{review_id}/approval-summary
```

Returns server-derived:

- final values
- corrections
- acknowledgements
- missing optional values
- current-policy comparison
- contact-address difference
- readiness

Approve

```
POST /api/v1/reviews/{review_id}/approve
```

Requires:

- Idempotency-Key
- If-Match

Request:

- approval_designation: CURRENT or HISTORICAL
- expected customer version
- expected current policy ID or null
- expected current-policy version token
- update_customer_contact_address boolean

The request does not contain final policy values.

Response:

- approved policy version
- derived designation
- previous Current summary
- contact update result
- audit event references
- post-approval actions

Success: 201

Repeated matching approval: 200

Conflicts:

- Review stale
- Review not ready
- Already approved
- Current policy changed
- Customer changed
- Unsupported action

57. Rejection API

POST /api/v1/reviews/{review_id}/reject

Requires:

- Idempotency-Key
- If-Match

Request:

- rejection reason
- note when Other
- expected customer version

Response:

- Rejected review
- document state
- customer impact
- available Technical Retry
- Reprocessing
- replacement-upload action

Conflict with approval:

409 ALREADY_APPROVED

Repeated identical rejection returns original response.

58. Retry and reprocessing API

58.1 Technical Retry

POST /api/v1/documents/{document_id}/technical-retries

Request:

- failed attempt ID
- expected active attempt ID or null
- paid-processing confirmation
- expected document version

Requires idempotency.

Errors:

- retry not eligible
- retry limit reached
- active attempt exists
- usage limit reached

Success: 201 new attempt.

58.2 Intentional Reprocessing

`POST /api/v1/documents/{document_id}/reprocessing-attempts`

Request:

- source attempt to reuse
- selected approved pipeline version
- expected active attempt
- paid-processing confirmation

Existing approved policy remains unchanged.

58.3 Stored Result replay

Demo-only route defined in Demo API.

58.4 Replacement upload

Uses signed upload authorization with `replacement_of_document_id`.

No ambiguous generic Retry route is provided.

59. Correction Review API

Start

`POST /api/v1/policy-versions/{policy_version_id}/correction-reviews`

Requires idempotency.

Creates an Open review with approved baseline.

Load and edit

Uses normal Review API.

Readiness and approval

Uses:

- Review readiness
- Approval summary
- Approve

Abandon

```
POST /api/v1/reviews/{review_id}/abandon
```

Conflict

If the customer's Current policy changes after Correction Review starts:

- review becomes stale
- approval returns `CURRENT_POLICY_CHANGED`
- reviewer must decide whether to restart from the newer version

No attempt or AI call is created.

60. Policy-version API

Current

```
GET /api/v1/customers/{customer_id}/current-policy
```

History

```
GET /api/v1/customers/{customer_id}/policy-versions
```

Version detail

```
GET /api/v1/policy-versions/{policy_version_id}
```

Returns:

- immutable approved fields
- named insureds
- vehicles
- drivers
- coverages
- reviewer
- approval date
- source document
- source attempt
- source review
- Current/Historical derived state
- correction/replacement relationships
- reviewer changes
- acknowledged warnings
- missing optional values

Comparison

GET /api/v1/policy-versions/{policy_version_id}/comparison

Query:

- other_policy_version_id

Used only for replacement or correction summaries.

No update endpoints exist for approved policy data.

61. Audit API

Global operational activity

GET /api/v1/activity

Approved for Overview recent activity and operational review.

Filters:

- category
- actor
- date range
- document
- customer
- cursor
- limit

Resource activity

- GET /customers/{id}/activity
- GET /documents/{id}/activity
- GET /policy-versions/{id}/activity

Response:

- event ID
- event type
- summary
- actor
- timestamp
- related links
- permitted structured before/after detail

No technical log, provider response, prompt, or stack trace is returned.

62. Usage API

```
GET /api/v1/usage/current
```

Response:

- period
- live-processing state
- documents processed
- pages processed
- OCR pages
- AI attempts
- repairs
- Technical Retries
- Reprocessing
- failures
- Stored Results
- estimated cost or unavailable
- configured limits
- remaining allowance
- approaching-limit state
- processing-disabled reason

No public Release 1 endpoint changes limits.

Owner configuration remains deployment or controlled operational data.

63. Acknowledgement-email API

Preview

```
GET /api/v1/policy-versions/{policy_version_id}/acknowledgement-preview
```

Returns fixed:

- recipient
- subject
- body
- content version
- demo simulation status
- prior sends

Request send

```
POST /api/v1/policy-versions/{policy_version_id}/acknowledgements
```

Requires idempotency.

Request contains no recipient, subject, or body.

Response: 202

Delivery status

```
GET /api/v1/acknowledgements/{email_delivery_id}
```

Resend

```
POST /api/v1/acknowledgements/{email_delivery_id}/resend
```

Requires:

- duplicate confirmation
- idempotency

Missing email:

```
409 CUSTOMER_EMAIL_REQUIRED
```

Provider failure remains independent of approval.

64. Demo API

Create or enter session

```
POST /api/v1/demo/sessions
```

Creates or returns an active disposable session.

Get context

```
GET /api/v1/demo/session
```

List samples

```
GET /api/v1/demo/samples
```

Start Stored Result

```
POST /api/v1/demo/samples/{template_id}/stored-replays
```

Requires idempotency.

Optional live sample

```
POST /api/v1/demo/samples/{template_id}/live-attempts
```

Requires:

- enabled feature
- remaining allowance
- explicit cost confirmation
- idempotency

Demo allowance

```
GET /api/v1/demo/usage
```

Reset

```
POST /api/v1/demo/reset
```

Requires idempotency and expected demo-session version.

May complete synchronously for small data or return `202` for background reset.

Repeated reset returns the same resulting generation.

Guided sample

```
POST /api/v1/demo/guided-sample
```

Returns or creates the recommended sample workflow.

Expiry

Expired session requests return:

```
410 DEMO_SESSION_EXPIRED
```

The frontend may create a new session.

65. Error contract

Envelope

```
{
  error: {
    code: "REVIEW_STALE",
    message: "This review changed after you opened it.",
    field_errors: [
      {
        field: "policy_number",
        code: "INVALID_FORMAT",
        message: "Check the policy number shown in the document."
      }
    ],
    action: {
      type: "REFRESH_REVIEW",
      label: "Refresh review"
    },
    retryable: false,
    replacement_required: false,
    paid_usage_possible: false,
    reference_id: "ERR-...",
    current_resource: {
      id: "...",
      version: "rv:18",
      state: "APPROVED"
    },
    limit: null,
    conflict: {
      expected_version: "rv:17",
      current_version: "rv:18"
    }
  }
}
```

Required codes

- UNAUTHENTICATED
- FORBIDDEN
- NOT_FOUND
- VALIDATION_FAILED
- DUPLICATE_CUSTOMER_WARNING
- DUPLICATE_DOCUMENT_WARNING
- UNSUPPORTED_FILE
- FILE_TOO_LARGE

- `TOO_MANY_PAGES`
- `LOCKED_PDF`
- `PROCESSING_LIMIT_REACHED`
- `ACTIVE_ATTEMPT_EXISTS`
- `RETRY_LIMIT_REACHED`
- `REVIEW_STALE`
- `REVIEW_NOT_READY`
- `ALREADY_APPROVED`
- `ALREADY_REJECTED`
- `CURRENT_POLICY_CHANGED`
- `CUSTOMER_STALE`
- `STORAGE_UNAVAILABLE`
- `PROCESSING_FAILED`
- `EMAIL_FAILED`
- `CUSTOMER_EMAIL_REQUIRED`
- `DEMO_RESTRICTED`
- `DEMO_SESSION_EXPIRED`
- `IDEMPOTENCY_CONFLICT`
- `OPERATION_IN_PROGRESS`
- `PIPELINE_VERSION_UNAVAILABLE`

Provider-specific messages and stack traces remain hidden.

66. Concurrency contract

Compared mechanisms

Updated timestamp

Easy but can collide and has ambiguous precision.

Integer row version

Simple and deterministic.

Arbitrary opaque token

Flexible but harder to inspect.

ETag with integer version

Combines standard HTTP semantics with deterministic database state.

Recommendation

Use integer `row_version`, exposed as an ETag-style opaque token.

Applied to

- Customer update
- Review corrections
- Field decisions
- Warning resolution
- Item changes
- Approval
- Rejection
- Current-policy replacement
- Correction Review
- Demo reset

Conflict response

- HTTP 409
- current resource version
- current state
- action guidance
- no partial mutation

Use 412 Precondition Failed only if the project later chooses strict HTTP cache semantics. Release 1 consistently uses 409 for business-state conflicts.

67. Idempotency contract

67.1 idempotency_records

Column	Type	Null
id	uuid	No
agency_id	uuid	No
demo_session_id	uuid	Yes
actor_scope_type	text	No
actor_scope_id	uuid	No
route_key	text	No
idempotency_key	text	No
request_fingerprint	text	No
status	text	No
response_status	integer	Yes

Column	Type	Null
response_body	jsonb	Yes
resource_type	text	Yes
resource_id	uuid	Yes
failure_code	text	Yes
created_at	timestampz	No
completed_at	timestampz	Yes
expires_at	timestampz	No

Unique:

(actor_scope_type, actor_scope_id, route_key, idempotency_key)

Statuses:

- IN_PROGRESS
- COMPLETED
- FAILED

Required operations

- Customer creation
- Upload authorization
- Upload completion
- Attempt creation
- Correction save
- Approval
- Rejection
- Technical Retry
- Reprocessing
- Email send
- Demo reset

Behavior

- Same key + same fingerprint + completed: replay response.
- Same key + same fingerprint + in progress: 409 OPERATION_IN_PROGRESS.
- Same key + changed fingerprint: 409 IDEMPOTENCY_CONFLICT.
- Failed before business transaction: may be safely retried with same key.
- Failed after terminal business result: record and replay actual result.
- Retain longer than likely client retry windows; exact retention is configurable.

Provider idempotency is additional protection, not the primary contract.

68. Background-job contract

68.1 Process Document Attempt

- Producer: Upload, Retry, Reprocessing
- Aggregate: Processing attempt
- Maximum internal executions: configured small number
- Terminal: candidate published or attempt failed
- Audit: start, success, final failure where meaningful

68.2 Replay Stored Result

- Producer: Demo API
- Aggregate: Stored Result attempt
- No provider calls
- Terminal: review created or template incompatibility

68.3 Send Acknowledgement Email

- Producer: Email API
- Aggregate: email delivery
- Internal retry only for temporary provider failures
- Terminal: Sent, Failed, or Simulated

68.4 Expire Demo Session

- Producer: scheduler
- Aggregate: demo session
- Terminal: cleanup pending or expired

68.5 Reset Demo Session

- Producer: Demo API when asynchronous
- Aggregate: demo session
- Terminal: new reset generation or failure

68.6 Cleanup Artifact

- Producer: retention scheduler or stage completion
- Aggregate: file/artifact
- Terminal: deleted or cleanup failed

68.7 Cleanup Upload

- Producer: upload expiry scheduler
- Aggregate: upload authorization
- Terminal: orphan removed

68.8 Detect Stale Attempt

- Producer: scheduler
- Aggregate: processing attempt
- Terminal: no action, requeue, or final failure

68.9 Release Usage Reservation

- Producer: scheduler
 - Aggregate: usage reservation
 - Terminal: released or finalized
-

69. Document-processing job contract

Payload fields

- attempt ID
- document ID
- source file ID
- expected pipeline version ID
- attempt origin
- processing mode
- Stored Result template ID when applicable
- usage reservation IDs
- expected active attempt ID
- reusable artifact IDs
- payload version
- correlation ID

The payload contains references, not provider secrets or raw document content.

Worker validation

Before work:

- job owns valid lease
- attempt exists and is nonterminal
- document active attempt matches
- source file exists
- pipeline version is available and approved
- job handler is compatible
- usage reservation remains valid
- Stored Result mode is consistent
- no newer attempt superseded it

Stage commits

Each stage:

- creates or reuses artifacts
- commits stage execution
- updates attempt progress
- updates job heartbeat
- records usage when applicable

Duplicate execution

A repeated worker execution reads completed stages and does not recreate them.

Source missing

Fail attempt as replacement-required when source no longer exists.

Pipeline unavailable

Fail with configuration mismatch. Do not silently select another model or pipeline.

Success

Candidate publication and attempt success occur transactionally.

Failure

Final attempt failure, document state, usage, and relevant audit event are committed together.

70. Stage-completion contract

Every stage completion records:

- stage execution ID
- attempt ID
- precondition state
- input artifact links
- output artifact links
- configuration identity
- status
- start and end
- duration
- usage record
- metrics
- diagnostic reference

- next-stage decision

Idempotency

A repeated completion with the same:

- attempt
- stage
- execution number
- configuration
- output hash

returns the existing completion.

A conflicting output for an already completed stage fails and requires investigation.

Failure

Stage failure records:

- category
- retryability
- preserved inputs
- paid-usage possibility
- diagnostic reference
- next recovery action

71. Stored Result replay job contract

Payload:

- demo session ID
- Stored Result template ID
- newly created attempt ID
- expected contract version
- reset generation
- correlation ID

Behavior:

1. Validate active session.
2. Validate template is published.
3. Validate contract compatibility.
4. Create stage executions using template stage plan.
5. Mark usage Stored.
6. Instantiate relational candidate.

7. Instantiate evidence.
8. Instantiate warnings.
9. Publish candidate.
10. Create real review.
11. update document state.
12. create audit event.

No OCR or AI call occurs.

Incompatibility fails the attempt visibly and blocks public release until the template is regenerated.

72. Email job contract

Payload:

- email delivery ID
- policy version ID
- content version
- recipient snapshot
- demo/live mode
- product idempotency identity
- provider idempotency identity
- correlation ID

Before sending:

- delivery is Pending
- policy exists
- recipient matches stored snapshot
- status is not already Sent
- demo mode is consistent

Success:

- mark Sent
- provider reference
- usage record
- audit event

Failure:

- mark Failed
- category
- diagnostic reference
- approval unchanged

Deliberate resend creates a new delivery record linked to the original.

73. Cleanup-job contracts

Expired upload cleanup

Scheduled and event-triggered after rejection.

Deletes orphan Storage object and marks authorization expired/cleaned.

Temporary artifact cleanup

Scheduled using artifact expiry.

Raw-response expiry

Scheduled.

Deletes content but preserves non-sensitive artifact metadata and attempt history.

Demo cleanup

Scheduled after session expiry.

Deletes disposable aggregates in controlled order and shared templates remain.

Abandoned claim recovery

Periodic stale-job detection returns eligible jobs to Available or finalizes dead jobs.

Usage reservation release

Scheduled after expiry and triggered by attempt cancellation/failure.

74. Transaction inventory

Operation	Main tables	Invariant	Failure outcome	Idempotency
Customer creation	customer, audit, idempotency	One business result	No customer	Yes
Upload completion	upload auth, file, document, attempt, usage reservation, job, audit	Processing work durable before Processing state	No active document workflow	Yes

Operation	Main tables	Invariant	Failure outcome	Idempotency
Attempt/job creation	attempt, document, reservation, job	One active attempt	Existing active returned	Yes
Correction save	correction, review, audit	Current review version	No partial correction	Yes
Warning resolution	resolution, review, audit	Valid warning and baseline	No resolution	Yes
Approval	policy and child tables, current selection, customer, review, document, audit, idempotency	Immutable version and one Current	Full rollback	Yes
Rejection	review, document, audit	One final outcome	Review unchanged	Yes
Current replacement	current selection, policy metadata, audit	One Current	Old Current retained	Included in approval
Email request	email, job, audit, idempotency	One send request	No job or delivery	Yes
Technical Retry	attempt, reservation, job, document, audit	Max two and one active	No new attempt	Yes
Reprocessing	attempt, reservation, job, document, audit	Approved data unchanged	No attempt	Yes
Demo reset	demo session, disposable aggregates, audit/job	Known complete generation	Old state retained or reset failed	Yes
Stage publication	stage, artifacts, attempt	Stage output once	Stage remains incomplete/ failed	Internal
Attempt success	candidate, warnings, review, attempt, document, usage, audit	Candidate and review consistent	Attempt not succeeded	Internal
Attempt failure	attempt, document, reservation, usage, audit	Final state consistent	Retriable transaction	Internal
Stored Result instantiation	attempt, stage, candidate, review, usage, audit	Same real workflow contract	No partial demo result	Yes

75. Query and page-loading requirements

Screen	Recommended loading	Main resources	Notes
Overview	One summary call + activity call	<code>/overview</code> , <code>/activity</code>	Add dedicated Overview resource
Customer List	One paginated call	<code>/customers</code>	Current-policy and attention projections
Customer Details	Customer call, tab-specific calls	customer, current policy, documents, history, activity	Independent tabs
Documents	One paginated call + status feed	<code>/documents</code> , status feed	Avoid per-row attempt queries
Processing Detail	Document + processing + source access separately	document, processing	Poll lightweight status
Review Queue	One paginated call	<code>/review-queue</code>	Cached readiness counts
Review Workspace	Review data and PDF access independently	<code>/reviews/{id}</code> , source access	Prevent PDF failure blocking fields
Approval Summary	One server-derived call	<code>/reviews/{id}/approval-summary</code>	No client assembly
Approved Summary	One policy detail call	<code>/policy-versions/{id}</code>	Read-only
Policy History	Paginated policy list	customer policy versions	Current derived
Audit	Paginated activity	activity resources	Load More
Usage	One period summary	<code>/usage/current</code>	No per-record calls
Demo Overview	Demo context + normal Overview	demo session, overview	Session-scoped

Additional endpoint

`GET /api/v1/overview`

Returns operational counts, recent approvals, usage state, and urgent items.

N+1 risks are avoided through relational projections and preloaded summaries.

76. Search, filter, sort, and pagination contract

Customers

Search:

- name
- email
- normalized phone
- current approved policy number

Filters:

- attention
- current policy yes/no

Default sort:

- `-last_activity_at`, then ID

Documents

Search:

- document name
- customer
- candidate or approved carrier
- candidate or approved policy number

Filters:

- stored disposition
- derived display category
- warning
- processing method
- date
- Stored/live

Default:

- `-last_activity_at`

Review Queue

Search:

- customer
- document

- carrier
- policy number

Filters:

- severity
- OCR
- current policy
- age

Default:

- severity descending
- ready/open time ascending
- ID

Audit

Filters:

- category
- actor
- date
- document

Default:

- occurred time descending
- ID descending

Matching

- Case-insensitive
- Partial matching
- Normalized email/phone
- No advanced syntax
- No fuzzy identity merge

Pagination

Cursor pagination for all growing lists.

Default 25.

Maximum 100.

Clearing filters returns to the first cursor.

77. Security and privacy contract

FastAPI authority

FastAPI is the primary business-authorization path.

Supabase RLS

Recommendation:

- Use RLS as defense in depth for private Storage.
- Do not expose business tables directly to browser roles.
- Backend-only business tables should deny direct anonymous/authenticated client access or remain outside exposed client schemas.
- Server credentials remain only in web and worker environments.

Object authorization

Every request verifies:

- agency
- demo session
- parent relationship
- actor status
- resource state

Raw artifacts

Accessible only to restricted technical operations, not normal staff APIs.

PII

- Avoid PII in error messages.
- Audit values are minimized.
- Logs use IDs and masked summaries.
- Signed URLs are never logged.
- Provider payloads are not recorded in normal logs.

Rate limiting

Required for:

- upload authorization
- upload completion
- retry
- reprocessing
- signed access
- email

- demo session creation/reset
- search endpoints

Demo abuse

- Curated samples only
 - session allowance
 - session expiry
 - live processing restricted
 - simulated email
 - isolated records
-

78. Database migration strategy

Ownership

One migration stream owned by the backend application.

Rules

- Prefer additive changes.
- Add nullable columns before requiring them.
- Backfill in controlled steps.
- Add constraints after compatible code is deployed.
- Avoid destructive column reuse.
- Preserve old candidate and pipeline interpretation.
- Constrained text values must support web/worker mixed-version windows.
- Web and worker releases declare compatible database version range.

Enum evolution

Because constrained text is used:

1. Expand check constraint.
2. Deploy readers tolerant of new value.
3. Deploy writers.
4. Remove deprecated values only after old jobs and records are handled.

Seed data

Migrations or controlled seed process create:

- agency
- approved pipeline versions
- email content version
- demo template sets

- Stored Result templates references

Synthetic data is not mixed with production seeds.

Rollback

Application rollback is preferred.

Destructive database rollback is avoided.

79. Compatibility and contract versioning

API version

`/api/v1`

Additive compatible fields do not create v2.

A new API version is required for incompatible semantics, not cosmetic additions.

Job payload version

Every job stores `payload_version`.

Worker rejects unsupported versions without guessing.

Candidate contract

Recorded on every candidate and Stored Result.

Evidence format

Versioned through evidence generator and geometry format.

Warning format

Versioned through warning-rules version and event explanation keys.

Audit details

`event_version` validates JSON details.

Pipeline configuration

Immutable pipeline version.

Frontend/backend compatibility

Backend may add fields.

Frontend must ignore unknown additive fields.

Removed or redefined fields require version coordination.

Web/worker compatibility

Job records include required handler version.

Stored Results

Regenerate when:

- candidate contract incompatible
- evidence format incompatible
- warning behavior materially changes
- template validator fails

80. Test and verification requirements

Contract tests must verify:

- Every FK rejects invalid ownership
- Unique active attempt
- One current policy
- One primary named insured
- Positive page and usage values
- Approval rollback
- Repeated approval
- Concurrent retry
- Worker claim exclusivity
- Expired lease reclaim
- Duplicate stage completion
- Demo cross-session denial
- Signed upload path restriction
- Signed download authorization
- Error envelope stability
- Pagination cursor stability
- Search/filter correctness
- Stored Result compatibility
- Idempotency fingerprint conflict
- Customer/review ETag conflicts

- Web/worker payload compatibility
 - Retention cleanup preserves business metadata
 - Raw artifacts are not returned by staff APIs
-

81. Data and API acceptance criteria

Identity

- Valid identity without membership cannot access business records.
- Demo identity accesses only its session.

Customers

- Customer requires only full name.
- Duplicate override remains auditable.
- Stale update cannot overwrite newer contact data.

Documents

- One source file maps to one document.
- Composite display state is derived consistently.

Uploads

- Browser cannot choose storage path.
- Repeated completion creates one document.
- Invalid PDF creates no paid attempt.

Attempts

- One active attempt per document.
- Previous attempts remain unchanged.

Jobs

- One worker holds a valid lease.
- Expired job can be reclaimed safely.

Artifacts

- Large bytes remain outside PostgreSQL.
- Retention and access are explicit.

Candidates

- Candidate cannot be updated after publication.
- Only approved fields exist relationally.

Evidence

- Evidence target and page belong to same candidate document.
- No exact geometry exists when unavailable.

Warnings

- Generated warning remains preserved after resolution.

Review

- Review mutations require current version.
- Readiness is server-derived.

Approval

- All effects commit or roll back.
- Repeated approval creates one version.

Policy versions

- Approved values cannot be edited.
- One Current policy is guaranteed.

Retries

- Third Technical Retry is rejected.
- Reprocessing remains distinct.

Audit

- Business events are append-only.
- Technical logs are not exposed.

Usage

- Concurrent reservations cannot bypass limits.
- Stored Results are not counted as live.

Email

- Arbitrary recipient or content is rejected.
- Failure does not change approval.

Demo

- Reset restores known state.
- Cross-session references are rejected.

APIs

- Every documented error uses the standard envelope.
- Every growing list is paginated.

Polling

- Lightweight status supports conditional requests.
- Final states stop polling.

Concurrency

- Expected-version mismatch returns conflict without mutation.

Idempotency

- Same request replays result.
- Changed request with same key fails.

Privacy

- Source text and raw provider output are absent from normal responses and logs.

Retention

- Expiring artifacts become cleanup jobs.
- Business records are not removed by diagnostic cleanup.

Migrations

- Web and worker remain compatible during additive rollout.

82. Main data and API risks

Risk	Likelihood	Impact	Mitigation	Residual risk	Blocks
Schema too generic	Medium	High	Fixed field contract and explicit tables	Some target polymorphism remains	Yes
Excessive JSONB	Medium	High	JSON only for snapshots/ geometry/ diagnostics	Flexible review values remain JSON	Yes
Excessive fragmentation	Medium	Medium	Balanced candidate and approved tables	More joins	No

Risk	Likelihood	Impact	Mitigation	Residual risk	Blocks
Candidate coupled to approved data	Low	Critical	Copy-on-approval tables	Mapping bugs	Yes
Current policy inconsistency	Low	Critical	Separate selection and transaction	Operational bug possible	Yes
Duplicate active attempts	Medium	High	Partial unique index	Recovery complexity	Yes
Lease bugs	Medium	High	Execution history and expiry tests	Long stages need heartbeat	Yes
Repeated approval	Medium	Critical	Unique source review and idempotency	Key misuse	Yes
Correction history unclear	Medium	Medium	Append + current flag	More rows	No
Ambiguous evidence target	Medium	High	Target contract and validation	Polymorphic target not FK-enforced	Yes
Demo leakage	Low	Critical	Session on aggregates and authorization tests	Coding mistakes	Yes
Idempotency growth	High	Low	Expiry cleanup	Retention tuning	No
Audit contains PII	Medium	High	Event contracts and masking	Material changes may contain PII	Yes
Raw artifacts exposed	Low	Critical	No staff API and restricted storage	Privileged operator access	Yes
Browser bypasses FastAPI	Medium	Critical	Deny direct table access	Misconfigured service key	Yes
Web/worker mismatch	Medium	High	Job versions and deploy checks	Mixed deploy window	Yes
Stored Result incompatible	Medium	High	Template validator	Old demo unavailable until regenerated	Public launch
Missing indexes	Medium	Medium	Screen-query review	Real data may expose new patterns	No
Too many indexes	Medium	Medium	Index only known queries	Some write cost	No

Risk	Likelihood	Impact	Mitigation	Residual risk	Blocks
Generic action RPC API	Medium	Medium	Named resources and distinct commands	Some command endpoints remain	No
One giant review update	Avoided	High	Granular writes	More API calls	No
Enum migration difficulty	Medium	Medium	Constrained text	Contract synchronization	No

83. Scope-creep findings

Generalized extraction-schema tables

Attractive for future document types but weakens exact Release 1 contracts.

Revisit after a second document type is approved.

Dynamic custom fields

Requires user configuration, validation, UI, and provider prompt generation.

Event sourcing

Auditability does not require rebuilding state from events.

CQRS

Read scale is insufficient to justify separate models.

GraphQL

REST resources match the fixed screens and business commands.

Public API

No external integration use case is approved.

Webhooks

No external event consumer is approved.

Redis now

PostgreSQL queue is adequate until sustained volume proves otherwise.

Kafka

No stream-processing requirement.

Database per demo session

Too expensive and operationally unnecessary.

General multi-tenancy

Public demo isolation is not a customer tenancy model.

Configurable roles

All normal staff have the same Release 1 abilities.

Soft deletion everywhere

Creates unnecessary filtering and inconsistent retention.

Data warehouse

No advanced analytics.

Document chat and vectors

Outside the golden path.

General notification center

Polling feed and temporary notifications are enough.

Provider-routing tables

Only one production provider per capability.

Arbitrary email templates

Fixed acknowledgement only.

Custom workflows

One approved document workflow.

Cryptographic audit ledger

Append-only application audit is sufficient for Release 1 claims.

Advanced policy comparison

Limited approval comparison is enough.

84. Open data and API decisions

Decision 1: Candidate relational hybrid

Recommendation

Approve relational candidate tables plus immutable JSON snapshot.

Alternative

JSONB-only candidate.

Data impact

More tables but stronger review and search.

API impact

Stable typed responses.

Complexity

Moderate.

Operational impact

Efficient field and list queries.

Migration impact

Contract changes require additive schema and snapshot-version support.

Codex effect

Tasks must map extraction contract into explicit candidate records.

Decision 2: One orchestration job

Recommendation

One processing job with durable stage executions.

Alternative

One job per stage.

Impact

Smaller queue and simpler recovery.

Trade-off

Longer worker lease and heartbeat responsibility.

Decision 3: Current-policy selection table

Recommendation

Separate pointer table.

Alternative

Status column or customer FK.

Impact

True immutable policy versions.

Trade-off

One additional table and join.

Decision 4: Concurrency tokens

Recommendation

Integer row version exposed through ETag-style tokens.

Alternative

Timestamps.

Impact

Predictable conflict detection.

Decision 5: Page text location**Recommendation**

Relational page metadata plus inline or Storage-backed artifacts based on size.

Alternative

All text in PostgreSQL or all text in Storage.

Impact

Balanced queryability and database size.

Decision 6: Raw-artifact expiry**Recommendation**

Configurable expiry columns with scheduled cleanup jobs.

Alternative

Immediate deletion or permanent retention.

Impact

Supports diagnosis while limiting PII retention.

Exact duration remains an owner decision after evaluation.

Decision 7: Benchmark results**Recommendation**

Version-controlled files and private artifacts, not production tables.

Alternative

Evaluation tables.

Impact

Lower scope and no dashboard.

85. Human supervision checkpoints

Customer versus policy data

Explain that contact data is editable and policy data is document-derived and versioned.

Inspect separate tables and approval mapping.

Document versus file object

Explain business lifecycle versus Storage object lifecycle.

Inspect signed upload and cleanup behavior.

Attempt versus job

Explain attempt is business processing history; job is worker execution control.

Inspect restart and duplicate tests.

Candidate versus review

Explain immutable AI output versus mutable human decisions.

Inspect that review writes never update candidate tables.

Review versus approved version

Explain proposals versus final immutable copy.

Inspect approval transaction.

One Current policy

Explain separate selection pointer.

Inspect concurrent replacement test.

Evidence

Explain field/item target, page, excerpt, and optional geometry.

Inspect candidate/evidence ownership tests.

Warning generation versus resolution

Explain immutable generated warning and separate human resolution.

Inspect preserved warning after acknowledgement.

Transactions

Explain why approval, upload completion, retry creation, and email request are atomic.

Inspect rollback tests.

PostgreSQL job claims

Explain leases, heartbeats, expiry, and reclaim.

Inspect two-worker tests.

Optimistic concurrency

Explain stale writes are rejected rather than locked.

Inspect two-browser review.

Idempotency

Explain one business result for repeated requests.

Inspect changed-payload conflict.

Demo isolation

Explain top-level session ownership and shared templates.

Inspect cross-session denial.

Usage reservation

Explain reserving before paid work.

Inspect concurrent limit tests.

Raw retention

Explain diagnostic value versus PII exposure.

Inspect expiry and access control.

API authorization

Explain Supabase identity versus FastAPI permission.

Inspect direct-object access denial.

Signed upload

Explain server-generated path and transactional completion.

Inspect arbitrary path rejection.

Signed download

Explain reauthorization and expiry.

Inspect expired URL renewal.

Web/worker compatibility

Explain versioned payloads and handler requirements.

Inspect incompatible-job failure.

86. Consolidated schema catalog

Table	Module	Purpose	State	Parent	Volume	Retention
<code>agencies</code>	Identity	One agency context	Mutable metadata	—	Very low	Permanent
<code>app_users</code>	Identity	Auth subject mapping	Mutable	—	Low	Permanent
<code>agency_memberships</code>	Identity	Business authorization	Mutable	Agency/user	Low	Permanent
<code>demo_sessions</code>	Demo	Disposable session boundary	Mutable	Agency/template	Medium	Expiring
<code>customers</code>	Customers	Contact record	Mutable	Agency/session	Medium	Business demo

Table	Module	Purpose	State	Parent	Volume	Retention
<code>upload_authorizations</code>	Documents	Signed upload lifecycle	Mutable terminal	Customer	Medium	Temporary
<code>file_objects</code>	Platform	Private Storage metadata	Cleanup mutable	Agency/session	High	Categorized
<code>documents</code>	Documents	Source business record	Mutable lifecycle	Customer/file	Medium	Business demo
<code>pipeline_versions</code>	Processing	Reproducible configuration	Immutable approved	—	Very low	Permanent
<code>processing_attempts</code>	Processing	Processing history	Terminal immutable	Document	Medium/high	Business
<code>background_jobs</code>	Processing	Durable work	Mutable lifecycle	Attempt/email/etc.	High	Diagnostic
<code>background_job_executions</code>	Processing	Claim history	Immutable	Job	High	Diagnostic
<code>processing_stage_executions</code>	Processing	Stage progress/history	Terminal immutable	Attempt	High	Business diagnostic
<code>stage_artifact_links</code>	Processing	Stage I/O links	Immutable	Stage/artifact	High	With attempt
<code>processing_artifacts</code>	Processing	Artifact metadata/content ref	Immutable content	Attempt	High	Categorized
<code>document_pages</code>	Processing	Page metadata	Immutable after processing	Document	Medium/high	Business
<code>extraction_candidates</code>	Extraction	Candidate header	Immutable	Attempt	Medium	Business

Table	Module	Purpose	State	Parent	Volume	Retention
<code>candidate_policy_fields</code>	Extraction	Candidate policy scalar fields	Immutable	Candidate	Medium	Business
<code>candidate_named_insureds</code>	Extraction	Candidate insured list	Immutable	Candidate	Medium	Business
<code>candidate_vehicles</code>	Extraction	Candidate vehicles	Immutable	Candidate	Medium/high	Business
<code>candidate_drivers</code>	Extraction	Candidate drivers	Immutable	Candidate	Medium/high	Business
<code>candidate_coverages</code>	Extraction	Policy-level coverage	Immutable	Candidate	Medium	Business
<code>candidate_alternatives</code>	Extraction	Conflicting values	Immutable	Candidate	Medium	Business
<code>candidate_evidence</code>	Extraction	Source support	Immutable	Candidate/page	High	Business
<code>candidate_alternative_evidence</code>	Extraction	Alternative evidence	Immutable	Alternative/evidence	Medium	Business
<code>validation_signals</code>	Extraction	Deterministic findings	Immutable	Candidate	High	Business evaluation
<code>generated_warnings</code>	Extraction	Pipeline warnings	Immutable	Candidate	Medium/high	Business
<code>warning_signal_links</code>	Extraction	Warning provenance	Immutable	Warning/signal	Medium/high	Business
<code>reviews</code>	Review	Editable review aggregate	Mutable terminal	Document/baseline	Medium	Business
<code>review_field_corrections</code>	Review	Scalar edit history	Append/current	Review	High	Business
<code>review_field_decisions</code>	Review	Not Present/difference	Append/current	Review	Medium	Business
<code>warning_resolutions</code>	Review	Human warning decisions	Mutable terminal	Review/warning	Medium	Business

Table	Module	Purpose	State	Parent	Volume	Retention
<code>review_vehicle_changes</code>	Review	Added/ removed vehicles	Mutable terminal	Review	Medium	Business
<code>review_driver_changes</code>	Review	Added/ removed drivers	Mutable terminal	Review	Medium	Business
<code>review_named_insured_changes</code>	Review	Added/ removed insureds	Mutable terminal	Review	Low	Business
<code>policy_versions</code>	Policy	Immutable approved header	Immutable	Customer/ document/ review	Medium	Permanent
<code>policy_version_named_insureds</code>	Policy	Approved insureds	Immutable	Policy version	Medium	Permanent
<code>policy_version_vehicles</code>	Policy	Approved vehicles	Immutable	Policy version	Medium	Permanent
<code>policy_version_drivers</code>	Policy	Approved drivers	Immutable	Policy version	Medium	Permanent
<code>policy_version_coverages</code>	Policy	Approved coverage	Immutable	Policy version	Medium	Permanent
<code>customer_current_policies</code>	Policy	Current pointer	Mutable	Customer/ policy	Low	Permanent
<code>audit_events</code>	Audit	Business history	Append- only	Agency/ resources	High	Business demo
<code>usage_limits</code>	Usage	Configured allowance	Mutable config	Agency/ session	Low	Period based
<code>usage_reservations</code>	Usage	Reserved capacity	Mutable terminal	Attempt/ limit	Medium	Report
<code>usage_records</code>	Usage	Actual measured usage	Immutable	Attempt/ email	High	Report
<code>email_deliveries</code>	Email	Delivery lifecycle	Mutable terminal	Policy/ customer	Low/ medium	Business

Table	Module	Purpose	State	Parent	Volume	Retention
<code>demo_template_sets</code>	Demo	Immutable seed version	Immutable	—	Very low	Template
<code>stored_result_templates</code>	Demo	Immutable replay template	Immutable	Template/file/pipeline	Low	Template
<code>idempotency_records</code>	Platform	Safe HTTP repeats	Mutable terminal	Actor/route	High	Expiration

87. Consolidated API catalog

Method	Route	Module	Purpose	Auth	Idempotency	Concurrency	Success	Key
GET	<code>/api/v1/overview</code>	Platform	Operational landing data	Staff/demo	No	No	200	For
GET	<code>/api/v1/customers</code>	Customers	List/search	Staff/demo	No	No	200	Val
POST	<code>/api/v1/customers/duplicate-check</code>	Customers	Duplicate warning	Staff/demo	No	No	200	Val
POST	<code>/api/v1/customers</code>	Customers	Create	Staff/demo	Yes	No	201	Duplicate warning
GET	<code>/api/v1/customers/{id}</code>	Customers	Detail	Staff/demo	No	No	200	Not
PATCH	<code>/api/v1/customers/{id}</code>	Customers	Edit contact	Staff/demo	Optional	Yes	200	Customer status
GET	<code>/api/v1/customers/{id}/current-policy</code>	Policy	Current policy	Staff/demo	No	No	200/204	Not
GET	<code>/api/v1/customers/{id}/policy-versions</code>	Policy	History	Staff/demo	No	No	200	—
GET	<code>/api/v1/customers/{id}/documents</code>	Documents	Customer documents	Staff/demo	No	No	200	—

Method	Route	Module	Purpose	Auth	Idempotency	Concurrency	Success	Key
GET	/api/v1/customers/{id}/activity	Audit	Customer activity	Staff/demo	No	No	200	—
POST	/api/v1/uploads/authorizations	Documents	Signed upload	Staff	Yes	No	201	Der res lim
POST	/api/v1/uploads/{id}/complete	Documents	Confirm upload	Staff	Yes	Yes	201	Dup inv
GET	/api/v1/documents	Documents	List	Staff/demo	No	No	200	—
POST	/api/v1/documents/duplicate-check	Documents	Duplicate check	Staff	No	No	200	Val
GET	/api/v1/documents/{id}	Documents	Detail	Staff/demo	No	No	200	Not
GET	/api/v1/documents/{id}/status	Processing	Poll status	Staff/demo	No	Conditional	200/304	Not
GET	/api/v1/documents/{id}/processing	Processing	Stage detail	Staff/demo	No	No	200	—
GET	/api/v1/documents/{id}/attempts	Processing	Attempt history	Staff/demo	No	No	200	—
POST	/api/v1/documents/{id}/source-access	Documents	Signed read	Staff/demo	No	No	200	Sto una
GET	/api/v1/documents/{id}/failure	Processing	Failure detail	Staff/demo	No	No	200	No
GET	/api/v1/documents/{id}/activity	Audit	Document activity	Staff/demo	No	No	200	—
GET	/api/v1/processing/status-feed	Processing	Combined polling	Staff/demo	No	Conditional	200/304	—

Method	Route	Module	Purpose	Auth	Idempotency	Concurrency	Success	Key
GET	/api/v1/review-queue	Review	Work queue	Staff/demo	No	No	200	—
GET	/api/v1/reviews/{id}	Review	Workspace	Staff/demo	No	No	200	Sta con sho
POST	/api/v1/reviews/{id}/corrections	Review	Save correction	Staff/demo	Yes	Yes	200	Rev
POST	/api/v1/reviews/{id}/corrections/{id}/undo	Review	Undo	Staff/demo	Yes	Yes	200	Rev
POST	/api/v1/reviews/{id}/field-decisions	Review	Not Present/ difference	Staff/demo	Yes	Yes	200	Inv dec
PUT	/api/v1/reviews/{id}/warnings/{warning_id}/resolution	Review	Resolve warning	Staff/demo	Yes	Yes	200	Res inv
POST	/api/v1/reviews/{id}/vehicles	Review	Add vehicle	Staff/demo	Yes	Yes	201	Val
PATCH	/api/v1/reviews/{id}/vehicles/{change_id}	Review	Edit added vehicle	Staff/demo	Optional	Yes	200	Sta
POST	/api/v1/reviews/{id}/vehicles/{vehicle_id}/remove	Review	Remove candidate vehicle	Staff/demo	Yes	Yes	201	Sta
DELETE	/api/v1/reviews/{id}/vehicle-removals/{change_id}	Review	Undo remove	Staff/demo	Yes	Yes	204	Sta
POST/ PATCH	/api/v1/reviews/{id}/drivers...	Review	Driver changes	Staff/demo	Yes	Yes	200/201	Sta
POST/ PATCH	/api/v1/reviews/{id}/named-insureds...	Review	Insured changes	Staff/demo	Yes	Yes	200/201	Sta
GET	/api/v1/reviews/{id}/readiness	Review	Approval blockers	Staff/demo	No	No	200	—

Method	Route	Module	Purpose	Auth	Idempotency	Concurrency	Success	Key
POST	/api/v1/reviews/{id}/refresh	Review	Refresh stale	Staff/demo	Yes	Yes	200	Already corrected
POST	/api/v1/reviews/{id}/abandon	Review	Abandon correction	Staff/demo	Yes	Yes	200	Not corrected
GET	/api/v1/reviews/{id}/approval-summary	Review	Final summary	Staff/demo	No	No	200	Not corrected
POST	/api/v1/reviews/{id}/approve	Policy	Approve	Staff/demo	Yes	Yes	201/200	Not currently changed
POST	/api/v1/reviews/{id}/reject	Review	Reject	Staff/demo	Yes	Yes	200	Already approved
POST	/api/v1/documents/{id}/technical-retries	Processing	Technical Retry	Staff	Yes	Yes	201	Retried
POST	/api/v1/documents/{id}/reprocessing-attempts	Processing	Reprocess	Staff	Yes	Yes	201	Action attempted
POST	/api/v1/policy-versions/{id}/correction-reviews	Review	Start correction	Staff/demo	Yes	No	201	Existing open
GET	/api/v1/policy-versions/{id}	Policy	Approved detail	Staff/demo	No	No	200	Not approved
GET	/api/v1/policy-versions/{id}/comparison	Policy	Limited comparison	Staff/demo	No	No	200	Invalid relationship
GET	/api/v1/policy-versions/{id}/activity	Audit	Policy activity	Staff/demo	No	No	200	—
GET	/api/v1/activity	Audit	Global activity	Staff/demo	No	No	200	—
GET	/api/v1/usage/current	Usage	Usage summary	Staff/demo	No	No	200	Usage unavailable

Method	Route	Module	Purpose	Auth	Idempotency	Concurrency	Success	Key
GET	/api/v1/policy-versions/{id}/acknowledgement-preview	Email	Preview	Staff/demo	No	No	200	Em req
POST	/api/v1/policy-versions/{id}/acknowledgements	Email	Request send	Staff/demo	Yes	No	202	Dup em req
GET	/api/v1/acknowledgements/{id}	Email	Delivery state	Staff/demo	No	No	200	Not
POST	/api/v1/acknowledgements/{id}/resend	Email	Deliberate resend	Staff/demo	Yes	No	202	Cor req
POST	/api/v1/demo/sessions	Demo	Enter demo	Demo	Yes	No	201/200	Dem una
GET	/api/v1/demo/session	Demo	Context	Demo	No	No	200	Exp
GET	/api/v1/demo/samples	Demo	Sample list	Demo	No	No	200	—
POST	/api/v1/demo/samples/{id}/stored-replays	Demo	Stored workflow	Demo	Yes	Yes	201	Inc tem
POST	/api/v1/demo/samples/{id}/live-attempts	Demo	Limited live run	Demo	Yes	Yes	201	Lim pro una
GET	/api/v1/demo/usage	Demo	Allowance	Demo	No	No	200	—
POST	/api/v1/demo/reset	Demo	Reset	Demo	Yes	Yes	200/202	Res
POST	/api/v1/demo/guided-sample	Demo	Guided workflow	Demo	Yes	No	200/201	—

88. Consolidated job catalog

Job	Producer	Responsibility	Aggregate	Retry	Idempotency	Termination outcome
<code>PROCESS_DOCUMENT_ATTEMPT</code>	Upload/ Retry/ Reprocess	Execute pipeline	Attempt	Limited internal	Attempt ID + pipeline	Candidate or Failure
<code>REPLAY_STORED_RESULT</code>	Demo API	Instantiate stored candidate	Attempt/ template	Limited technical	Session + template + request	Review Failed
<code>SEND_ACKNOWLEDGEMENT_EMAIL</code>	Email API	Send fixed message	Email delivery	Temporary provider failures	Delivery ID	Sent/ Failed/ Simulated
<code>EXPIRE_DEMO_SESSION</code>	Scheduler	Mark expired	Demo session	Limited	Session + expiry generation	Expired
<code>RESET_DEMO_SESSION</code>	Demo API	Replace disposable state	Demo session	Limited	Session + reset generation	Active/ generated/ Failed
<code>CLEANUP_ARTIFACT</code>	Retention process	Delete expired artifact	Artifact/file	Limited	Artifact ID + expiry	Deleted/ Failed
<code>CLEANUP_UPLOAD</code>	Scheduler	Delete orphan upload	Upload authorization	Limited	Authorization ID	Clean/ Failed
<code>DETECT_STALE_ATTEMPT</code>	Scheduler	Recover or fail stale work	Attempt/job	Controlled	Attempt + observation window	Request/ Failed/ op
<code>RELEASE_USAGE_RESERVATION</code>	Scheduler/ business event	Release expired reservation	Reservation	Limited	Reservation ID	Released/ Finalized

89. Decision log

Decision	Reason	Alternative	Trade-off	Status	Revisit	Implementation effect
PostgreSQL relational truth	Integrity and queryability	Document DB	More schema design	Approved	No near-term	Migrations first
UUID public PKs	One opaque identity	Integer + public UUID	Larger indexes	Made here	Very high scale	Consistent contracts
Constrained text states	Migration flexibility	DB enums	App/DB list coordination	Made here	Stable schema	Check constraints
Candidate relational hybrid	Review/search plus reproducibility	JSONB-only	More tables	Owner approval	Second document type	Explicit mapping
Separate file object	Storage lifecycle	File columns on document	Extra table	Made here	None	Signed flows
One orchestration job	Simplicity	Per-stage jobs	Long lease	Owner approval	Higher volume	Durable stages
Separate current-policy pointer	Preserve immutability	Status on version	Extra join	Owner approval	None expected	Transactional selection
Integer ETag concurrency	Deterministic conflicts	Timestamp	Version increments	Owner approval	Cross-service need	If-Match
Granular review writes	Clear audit/conflicts	Full review JSON update	More endpoints	Made here	High-latency clients	Bounded tasks
Generated warnings immutable	Preserve AI behavior	Mutable warning	Resolution table	Made here	None	Separate resolution
Demo FK on top-level aggregates	Isolation without tenancy	Schema/database per session	Mandatory filters	Made here	Multiple customer tenants	Demo guards

Decision	Reason	Alternative	Trade-off	Status	Revisit	Implementation effect
Evaluation files outside product DB	Avoid dashboard scope	Evaluation tables	Less in-app querying	Owner approval	Larger evaluation team	Version-controlled results
Provider-neutral version IDs	Evaluation not final	Provider columns everywhere	Adapter mapping	Approved baseline	Provider winner	No routing schema
FastAPI authority, RLS defense	Central business rules	Direct browser DB	More backend APIs	Approved	None	Deny client tables
Final provider selection	Must follow benchmark	Preselect model	Evaluation delay	Pending evaluation	After spike	Roadmap starts with spike

90. Handoff requirements for the implementation roadmap

Document 7 must receive:

Schema

- Approved table catalog
- Ownership boundaries
- Candidate hybrid decision
- Current-policy pointer
- Demo isolation
- Retention categories

API

- Approved endpoint catalog
- Error envelope
- Pagination
- ETag concurrency
- Idempotency rules
- Signed upload and download sequence
- Polling resources

Jobs

- Approved job catalog
- One orchestration-job decision
- Lease and heartbeat rules

- Stage commit rules
- Worker compatibility
- cleanup schedules

Migration order

1. Identity and agency
2. Customer
3. Storage and upload
4. Pipeline configuration
5. Document and attempts
6. Jobs and stages
7. Artifacts and pages
8. Candidates
9. Evidence, validation, warnings
10. Reviews and corrections
11. Policy versions and Current selection
12. Audit
13. Usage
14. Email
15. Demo templates and sessions
16. Idempotency
17. Indexes and final constraints

Transaction boundaries

- Customer creation
- Upload completion
- Attempt creation
- Review mutations
- Approval
- Rejection
- Retry/Reprocessing
- Email request
- Stage publication
- Demo reset

Test requirements

- Constraints
- Job claims
- Idempotency
- concurrency
- approval rollback
- demo isolation
- private file access
- Stored Result compatibility

Provider evaluation spike

Implementation must begin with the controlled Document AI evaluation before final live provider integration.

Environment dependencies

- Supabase Auth
- PostgreSQL
- private Storage
- Render web and worker
- Vercel frontend
- Resend after email phase
- synthetic demo environment

Demo requirements

- Immutable template versions
- per-session seeding
- Stored Result replay
- reset generation
- usage simulation
- no real email

Owner checkpoints

- candidate storage
- job style
- Current policy strategy
- concurrency token
- raw retention
- benchmark-record location
- evaluation results
- provider selection

91. Recommended next planning document

The final pre-coding document should be:

Release 1 Implementation Roadmap and Codex Governance Plan

It should define:

- implementation phases
- controlled Document AI evaluation spike
- milestone order
- vertical slices
- repository initialization
- environment setup
- migration sequence
- backend foundation
- frontend foundation
- worker foundation
- Document AI evaluation
- provider selection checkpoint
- approved provider integration
- upload workflow
- customer workflow
- review workflow
- approval and immutable policy versions
- audit and usage
- email
- demo environment
- contract testing
- end-to-end testing
- deployment
- observability
- Loom-ready polish
- Codex task boundaries
- diff-review process
- human approval gates
- rollback rules
- decision-log updates
- interview-learning checkpoints

Implementation should begin only after the owner approves these major open decisions:

1. Candidate relational-hybrid design
2. One orchestration job per processing attempt
3. Separate Current-policy selection table
4. Integer ETag-style concurrency
5. Hybrid page-text artifact storage
6. Configurable raw-artifact expiry
7. Benchmark results stored outside the production database

The AI model, OCR provider, input representation, and final cost ceilings remain unresolved until the evaluation spike passes the approved benchmark gates.